Galaxy Raiders

Engineering Journal

Joshua Yam, Henry Low, Sim Xin Yu

Contents

1. Software Documentation
    a. General Approach
    b. Code Structure
    c. Sensor Libraries
    d. Odometry
    e. Localisation
    f. Complementary Filter
    g. Camera Tracking
    h. Path Following
    i. Debugging / Telemetry
2. Hardware Documentation
    a. Reducing Robot Base
    b. Fixing Ackermann Steering
    c. Other Design Considerations
    d. PCB
3. Power and Sensors
    a. Microcontroller
    b. Power
    c. Sensors
4. Design History

Software Documentation

General Approach

Both challenges are solved using non-”state machine” approaches because most sections of the programme — which is mainly getting out of parking, determining initial position, navigating through playfield and parking — happen sequentially, never looping or switching between each other.

Additionally, throughout most of the programme, we have a non-blocking loop to constantly keep track of and update the robot’s pose by reading from the gyro, lidar and encoder sensors. Here is a breakdown of the loop:

1. Read sensors
2. Estimating pose using encoder and gyro through odometry
3. Localisation using Iterative Closest Points (previously Spike Landmark) on LiDAR readings before merging the localised pose with the odometry pose using a complementary filter
4. Navigating using the new merged pose
5. Controlling servos and motors

Code Structure

In 2025, our programme was much messier because of the way we switched between open and obstacle modes. Specifically, we turned large sections of the programme into functions to import to the main file, where we would check the duration of the button press to determine which functions to run. Moreover, much of the main code could have been turned into individual functions, on top of the fact that the way we categorised the functions were ambiguous and clear, such that certain functions could have fit into more than one library / module. It is why in 2026, we completely refactored the code:

Open

```
initialise_hardware (initialise_hardware.py)
        gyro
        lidar
        encoder
        motors

wait for button press

find initial position (initialise_pos.py)
        read lidar
        calculate initial pos
```

```
intialise_telemetry (telemetry.py)
        initialise network connection

first path while True:
        If width is 1000, take path with x = 500
        elif width is 600, take path with x = 300
        check while moving
        if east > some dist:
                confirm pose
                break
        elif west > some dist:
                confirm pose
                break

main while True:
        read sensors (sensors.py)
                gyro
                lidar
                encoder

        estimate position (odometry.py)
        localise - lidar (spike_localisation.py, point_cloud_localisation.py) - same functions, input (estimate_pos,
lidar_readings) and output (localised_pos)
        merge estimate_pos and localised_pos (filter.py)
        follow paths (navigation.py)
        if end of a path:
                check spike position to check if next wall is extended / aim at a certain point to check distance
        if at ending position:
                break
stop
```

Obstacle

```
initialise_hardware (initialise_hardware.py)
        gyro
        lidar
        encoder
        motors

wait for button press

# Optional - start from parking lot
- Opening on left - CCW, right -- CW
find initial position (initialise_pos.py)
        read lidar
        calculate initial pos
                front dist > back dist (clockwise); back_dist > front_dist (CCW)
                if there are invalid readings (check with a range)
                        use only the valid reading

intialise_telemetry (telemetry.py)
        initialise network connection

if obstacle in front:
        move back

main while True:
        read sensors (sensors.py)
                gyro
```

```
                    lidar
                    encoder

        estimate position -- odometry using encoder and gyro (odometry.py)
        localise - lidar (spike_localisation.py, point_cloud_localisation.py) - same functions, input (estimate_pos,
lidar_readings) and output (localised_pos)
        merge estimate_pos and localised_pos (filter.py)
        follow paths (navigation.py)
        if end of a path:
                check camera for obstacle
        if at ending position:
                break

parking

stop
```

Common sections in both challenges

Under the initialisation stage are functions for starting up the sensors and finding the robot's position / direction. For easy reference to the sensors and up to date data, we put all the sensors into an object called sensors and their readings into an object called sensor_readings. Afterwards, we moved on to the main loop, which consists of the non-blocking loop.

Obstacle-specific programme

Determining initial position

The robot will determine its direction (clockwise or counterclockwise) by getting the front, rear and side distances because the initial position is generally fixed. It will check if an obstacle is in front of it before moving back.

Main run: avoiding obstacles

In the main run, the robot will switch between two fixed sets of paths, an outer and inner lane. Depending on the colour of the blocks in front, it will switch between the two paths to follow. For example, if the robot is moving clockwise, detecting a red block will cause it to follow the inner paths, and vice versa for the green blocks. This will switch if the robot is moving counterclockwise.

We also implemented camera tracking such that the camera follows the colour of the obstacle two sections ahead which ensures that the robot never misses the obstacle. In 2025, a major issue we had is that because our robot was larger and the field of vision of the robot was wider, we had to keep readjusting our paths so that the robot would be facing the obstacles directly to ensure that we did not miss it. As a result, we had to make so many adjustments to our paths: on the one hand, we had to ensure the obstacles fall within the robot's field of vision; on the other hand, the robot is

unable to make sharp turns so it needs enough space to steer. Now, with the camera tracking, we only have to focus on the robot's steering limits when creating the path as the camera will also follow the obstacle that is 2 paths ahead of the current path.

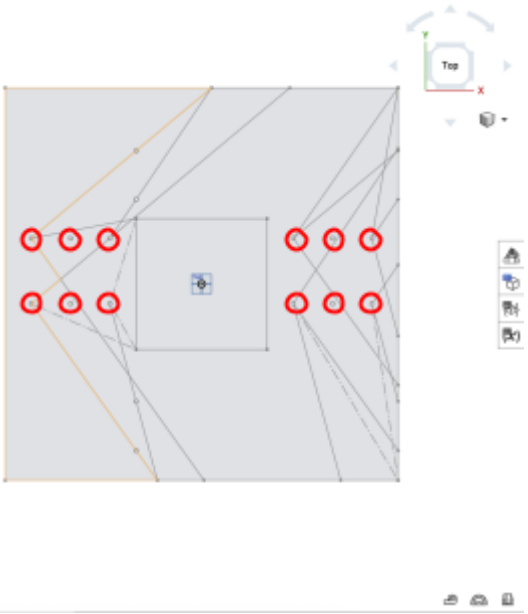
Parking

To get into the exact position at the start of parking, the robot repeatedly moves back and forth until the x, y values of the robot are within a certain range. Subsequently, the robot executes three turns before shifting back and forth again to wiggle itself into the parking area until its distance from the outer vertical wall is less than 20cm.

Open-specific programme

Determining initial position

Our final approach was for the robot to assume it is moving clockwise and travel the first path until it reaches the corner: this is where the robot will read the distances from the lidar sensor for the points (0mm, 2700mm) and (1000mm, 2700mm). If the distance for the left point is greater than the right point, it will deduce that it is moving counterclockwise and update its pose accordingly. The reason why we chose these points instead of measuring the left and right distances is because we encountered some issues with determining the sides at certain start positions. This was far more accurate than the approaches we used in 2025:



Used Onshape to visualise the possible positions (represented by dots which are circled in the above diagram)

Method 1:

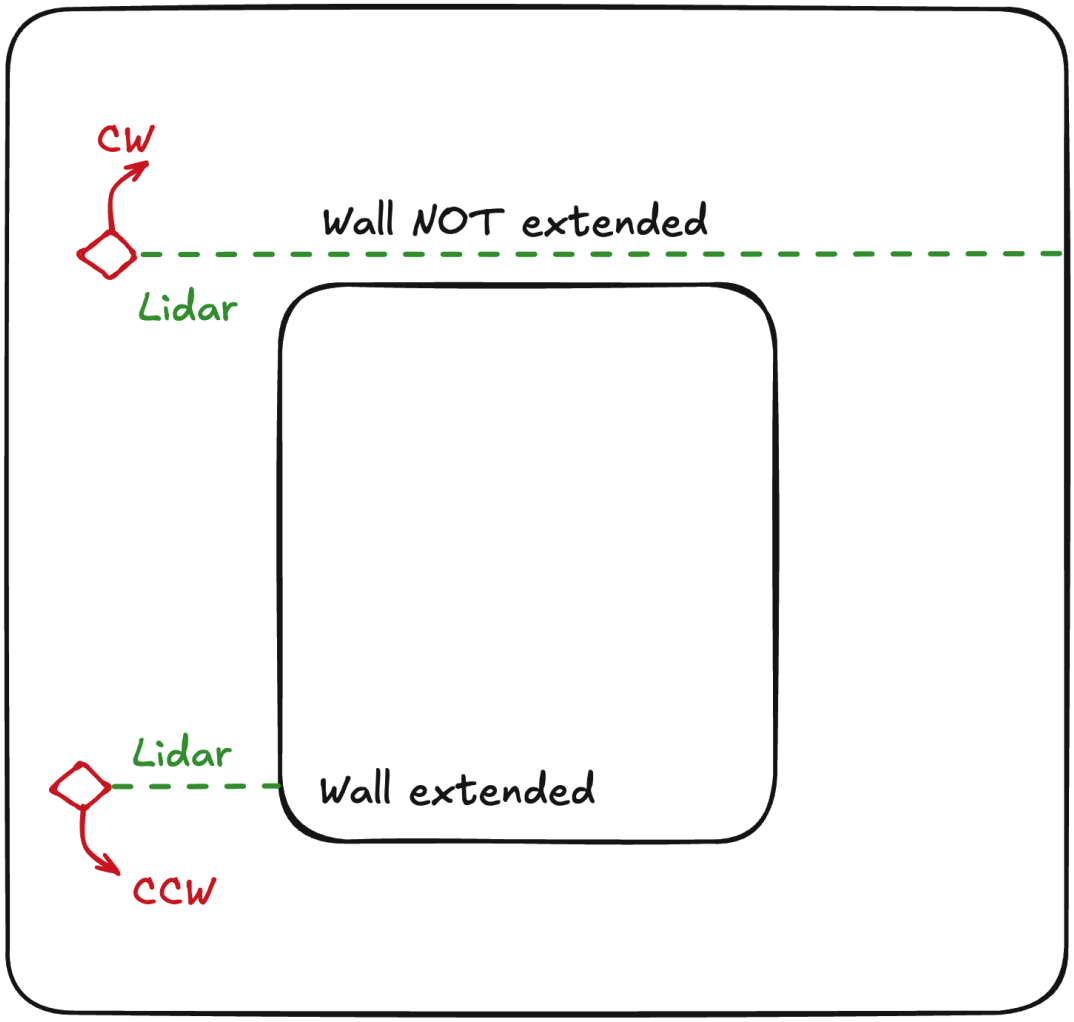
- The lines represent the lidar readings from the corresponding angles from each starting position
- These lidar readings are used to determine if the bot is on the left or right side of the field
    - A threshold is set for the different lidar values of each starting position from each angle, and if the lidar reading falls between the threshold, the starting position is then determined
- **Theory behind this**: we only need to determine if the bot is on the left or right side of the field, as the top and bottom of the field is identical to the left and right, just in different orientation
- Pros:
    - The exact starting position can possibly be determined (starting position 1, 2, 3, 4, 5, or 6 from top left to bottom right)
- Cons:
    - Was not very reliable, struggled to determine the correct side near the inner corners
    - As the lidar readings needed to be at precise angles, it sometimes had inaccurate readings due to the angle being slightly off
    - This method would not work in the Open Challenge when the inner walls are extended outwards, causing the lidar readings to be different, resulting in the bot detecting an inaccurate side

Method 2: (used during 2025)

- By detecting the inner spikes (corners of the inner square of the field)
- Using the inner spike readings, we can determine which side of the field (left or right) the bot is starting in
- After determining the side of the starting position, the exact x and y position of the bot can be determined by using the lidar readings for every 90° (0°, 90°, 180°, 270°)
    - This is done by calculating the average of the lidar reading from opposing sides, e.g. [(3000 - front_distance) + rear_distance] / 2 -> this calculates the y value of our robot
    - A special case would be when the x value is calculated, where the x value differs depending on which side the bot is starting in
        - E.g. if the bot starts on the left, the x value would be $0 < x < 1000$. If the bot starts on the right, the x value would be the same value as on the left side + 2000 (this is because the difference between the x value of the left and right starting position is always a constant of 2000cm)
- Pros:
    - Inner spikes are much easier to detect as compared to the outer spikes/walls from all 12 possible starting positions
    - This method works during the Open Challenge when the inner walls are extended, as the bot can still detect the inner spikes of the extended inner wall (as we have tested)
- Cons:
    - Still very unreliable as the robot might detect more spikes on the outer walls compared to the inner walls

Main run

The robot follows preplanned clockwise and counterclockwise inner and outer paths. The inner paths are 20cm away from the inner wall, while the outer paths are 70cm away from the inner wall. By default, it would take the outermost path. However, every time it finishes following a path, it will check the right (when clockwise) or left (when counterclockwise) distance of the lidar reading. If the distances are more than 80cm, it means the wall was not extended so it will switch to the inner path.



Previously, we used a safe but unintelligent approach, where the robot only travelled the outer paths, meaning it was far less efficient and spent more time navigating through the playfield.

Sensor Libraries

LiDAR - COIN-D4

We created our own library, because there weren't any python raspberrypi libraries, to receive data from the COIN-D4 LiDAR which uses the UART protocol. We used the serial package to open the

serial port /dev/ttyS0 with the baud rate 230400. The scan frequency is roughly 10 rotations per second, 16 frames per rotation, 25 readings per frame, which sums up to 16 x 25 = 400 readings in 0.1 seconds.

Everytime the input buffer is populated, we check if the first two characters match the data header '\xAA\x55' before retrieving the sample count (4th byte). Then, it reads until the pointer is at the final byte which would be: heading length + 3 bytes * sample_count.

After checking if the checksum is valid, we parse the frame, assigning the appropriate angles with their distances. However, we noticed that we were losing data after plotting out the lidar points that were being used:

With buffering

Bytes remaining in lidar - 344 → roughly 4 frames (85 bytes per frame)
Localisation count 48
Loop count 141
Loops per second 141/ 5 = 28.2

1000 ms / 28.2 = 35.5 ms for each loop, which includes

- Readings sensors
- Localisation
- Camera reading

Without buffering

Bytes remaining – 4095 / 85 = 48 frames
Localisation count 10
Loop count 142
Loop per second 142 / 5 = 28.4

It means we were receiving 48 / 160 = 0.3 seconds worth of outdated data every 5 seconds. Thus, in the _parse_frame() function, we used a double buffering approach, which has two empty frames initially. This allows the programme to fill one scan while processing the previous completed scan. When start_angle wraps back to a smaller value, _parse_frame() switches measurement_ptr to the other list and clears that list. New samples go into the new active list, while the completed scan remains available in the other list, thereby avoiding clearing or overwriting data the programme may still be reading.

Gyro - MPU-6050

The gyro sensor uses the i2c protocol so initially, we used the pigpio library to receive data, but realised it was too slow and caused significant delays, so we switched to smbus.

Additionally, we had to numerical integrate the z-axis gyroscope data over time to calculate the heading:

- Read the raw angular velocity
- Subtract the calibrated gyro offset value
- Calculate the time passed since the previous reading
- Take the sum of the current and previous z readings, multiply it by the time passed, and divide it by the sensitivity scale factor (131 * 2)
- To find the calibrated gyro offset value, we also created a calibration function so that when we leave the robot as stationary, we can take the average of 500 measurements (which are supposed to be zero values) to find the z error. We also created functions to save the calibration as a txt file and load it.

Encoder - AB Hall-effect Encoder

It is a type of quadrature encoder that measures rotation using the strength of the magnetic field in the x and y direction, and measures the positive/negative magnetic charge in those directions. The true mechanical angle is then recovered by applying arctangent because the rotating magnet produces orthogonal sine and cosine magnetic field components.

We used the pigpio library and programmed two callback functions to detect changes in states for each channel A (Pin 5) and B (Pin 6)

```
cb1 = pi.callback(5, pigpio.RISING_EDGE, self.step_count)
cb2 = pi.callback(6, pigpio.EITHER_EDGE, self.drive_dir)
```

In the first callback, when channel A rises, it checks the state of channel B. If B is high, it is moving backward, whereas if B is low, it is moving in the opposite direction, forward. The second callback is constantly updating the state of channel B.

Previously, we only read channel B when channel A goes from low to high. However, since the pipgio library would put that task in a cue, the time it takes for the raspberrypi to receive the signal the

change in state of channel A is delayed, leading to an incorrect reading of the motor's direction. Recording the change in states of both channels solves this issue by preserving the order of the changes in the states of the channels.

Encoder calibration:

2025

1st attempt: 1522.4mm – 5113 steps, 0.298mm – 1 step

2nd attempt: 4670 mm – 15795 steps, 0.296mm – 1 step

2026

Actual	Encoder	Time	Notes
1285	2705	5 s	
2005	4370	8 s	Discarded
2075	4365	8 s	
2070	4357	8 s	

Motor Driver - TA6586

The TA6586 is a monolithic, bidirectional H-bridge motor driver IC that uses sign-magnitude drive. In sign-magnitude control, the sign determines the motor's direction, while the magnitude determines how strongly the motor is driven. In the library, the sign of the speed argument selects the direction of rotation, while the absolute value of speed determines the PWM duty cycle and by extension the motor speed.

For positive speed, the GPIO pins 20, 21 are configured as such

self.pi.set_PWM_dutycycle(21, 255-speed) # PWM controlled
self.pi.set_PWM_dutycycle(20, 255) # constantly high
vice versa for negative speed

self.pi.set_PWM_dutycycle(21, 255) # constantly high
self.pi.set_PWM_dutycycle(20, 255+speed) # PWM controlled

This allows for lower switching losses because fewer transistors switch simultaneously

Camera - Raspberry Pi Camera Rev 1.3

We use the Raspberry Pi Camera Module with the picamera2 and cv2 library to identify the colour of the obstacles in front of the car.
The colour detection works as follows:

1. Configuration:
    - Defines specific boundary values for green (Hue: 44–75, Saturation: 84–255, Value: 41–255) and red (Hue: 112–128, Saturation: 90–255, Value: 93–255).
    - Blob detector parameters are configured to target white blobs (blobColor = 255), enforce area constraints (minArea = 650, maxArea = 100000), and set geometric boundaries like circularity (minCircularity = 0.1) and convexity (minConvexity = 0.87).
2. Frame capture:
    - A default 640x480 frame is retrieved from the camera array using picam2.capture_array("main").
    - The frame array is cropped via frame[105:-1, 0:-1] to remove the top 105 pixels, preventing the camera from seeing over playfield walls.
    - The cropped image is passed through cv2.GaussianBlur(frame, (5,5), 0) to smoothen uneven surfaces and reduce high-frequency noise.
    - The smoothed image is converted from default BGR to HSV format using cv2.cvtColor(frame, cv2.COLOR_BGR2HSV).
3. Masking
    - Using cv2.inRange(), a binary mask is applied to the image, where the white parts of the image represent the areas that are within the tested thresholds for the green and red obstacles and everything else turns black
    - The detector.detect() function scans both masks independently to locate valid shapes, returning keypoints that contain coordinate and size data.
4. Filtering:
    - Run cv2.SimpleBlobDetector() on the binary masks. It rejects blobs that are smaller than 650 pixels to remove any noise
    - Iterates through all detected green and red keypoints to isolate the largest respective blob (g_largest_keypoint and r_largest_keypoint).
    - Decision Logic:
        i. Returns None if neither color is detected.
        ii. Returns "g" or "r" if only one color is found.
        iii. Compares physical sizes (keypoint.size) if both are present, returning the character corresponding to the larger blob.

Servos (Camera Swivel, Steering) - MG90

To calibrate the servos, we tested with different values from 500 to 2500 microseconds (µs) to find the centre, maximum and minimum.

Servo	Centre	Max, Min (in degrees)
Camera	1475	87
Steering	1425	42

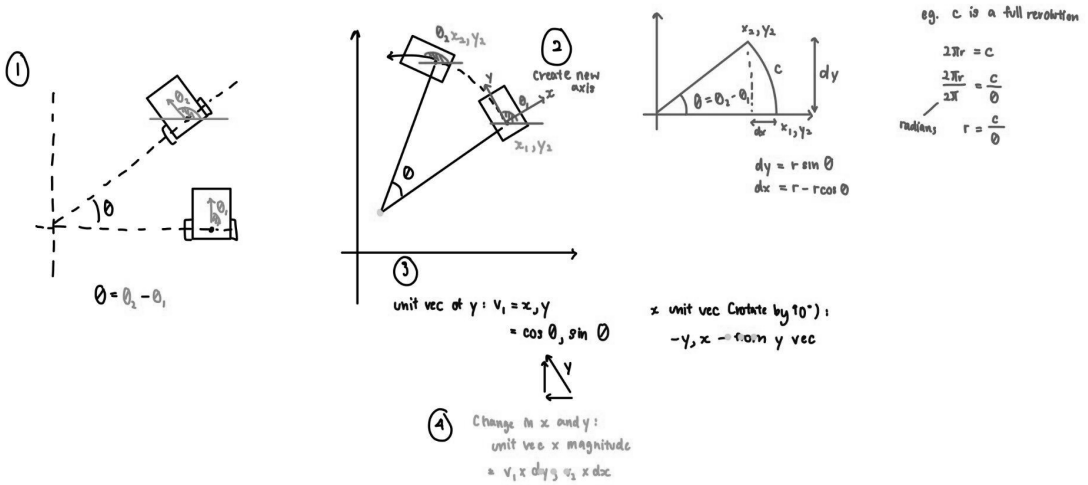
I/O Interface (Button, LEDs)

In total, we have a button to start and stop the programme and three LEDS (red, yellow, green) to signal different messages for different challenges: open (red -- stationary, green -- running), obstacle (yellow -- stationary, green -- sees green obstacle, red -- sees red obstacle).

Odometry

We use the wheel rotation and gyro to estimate the robot's pose (position and heading) while it is running.

- steer_in_dir(): steering (proportional control)
- augment_path(), augment_paths(): Steering back to line
    - Find path vector = (x2-x1) / (y2-y1)
    - Find distance of path
    - Find path vector in units
    - Find perpendicular path vector
    - Find path direction
- dot(): dot product
    - how far robot is off the path – drive_path()
    - how far robot has traveled along the path
- drive_path(), drive_paths():
- estimate_pose():
    - First version: looked at the heading and assumed a straight path based on the heading
    - Second version: treat the movement as an arc of a circle



First, we take the centre of rotation as the origin, which means

$$\theta = \theta_2 - \theta_1$$

$$d_y = r sin\theta, d_x = r cos\theta$$

$$r = \frac{c}{\theta}$$

in which c is the distance travelled

$$d_y = \frac{c}{\theta} sin\theta, d_x = \frac{c}{\theta} cos\theta$$

To apply this back to the original axis, we take the x, y vectors of the "morphed" graph, (rotate y vector by 90°), before we multiply the unit vectors with the local dx, dy to find the global dx, dy

$$y' = (cos\theta, sin\theta)$$

$$x' = (-sin\theta, cos\theta)$$

$$d_X = x' \times d_x, d_Y = y' \times d_y$$

Afterwards, we integrate other localisation methods on top of using odometry, by using a complementary filter.

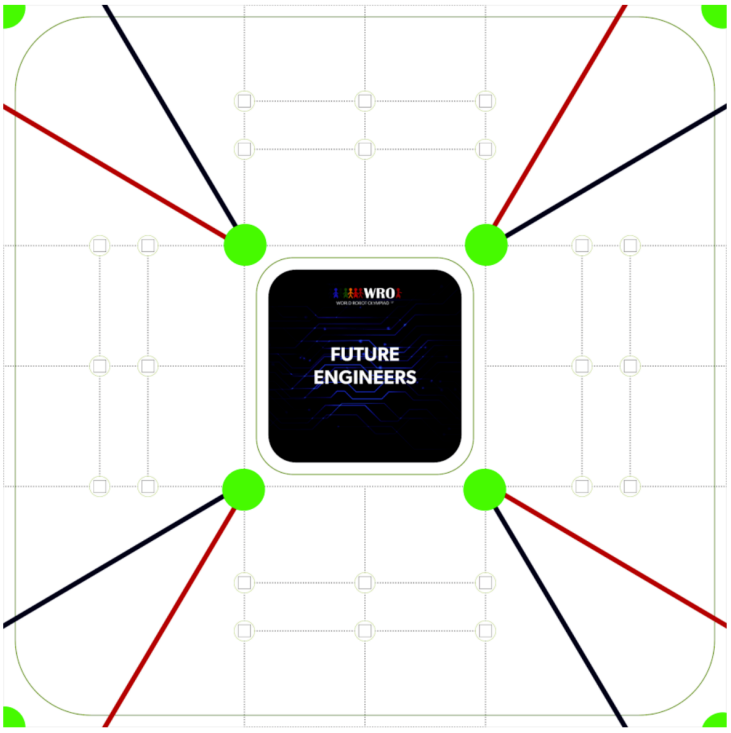
Localisation

Spike Landmark (Old)

Initially, we used the spike landmark method during the 2025 season of WRO, but we wanted to improve our localisation method so we have since shifted to the Iterative Closest Point method.

Essentially, the method identifies which lidar readings are spikes or points of interest, in our case corners which are noticeably further away than the adjacent readings. Afterwards, we calculate if there is any difference between the supposed and actual cartesian coordinate of the landmark to find the error.

- Defined 8 landmarks: 4 inner corners, 4 outer corners, marked as green circles



- add_cartesian(): convert lidar readings into cartesian coordinates
- identify_spikes()
    - the different between two lidar measurements adjacent to the centre measurement have to show a significant difference
    - filter out spikes within 250mm

- match_landmarks(): within 120mm, which we tested as the most optimal threshold
- calc_position_error(), calc_angle_error(): according to the spikes matched, we calculate the total error accumulated for x, y and heading (angle of the spike from the robot) separately before finding the average for each value and adding it to the odometry_pose

However, this method only uses a few points out of the approximate 400 we get from the lidar in comparison to the Iterative Closest Point method that makes full use of all the lidar readings.

Iterative Closest Point (current)

- augment_walls(): Define the 8 walls on the playfield, getting their path vector, path direction, unit path vector and perpendicular path vector which are values needed
- add_cartesians(): same as the one used in the Spike Landmark

localise(): Let's start by transforming 1 point on a cartesian plane.

To handle the rotation, we can

$$x_2 = R\cos(\alpha+\beta) = R[\cos(\alpha)\cos(\beta) - \sin(\alpha)\sin(\beta)]$$

$$y_2 = R\sin(\alpha+\beta) = R[\sin(\alpha)\cos(\beta) + \cos(\alpha)\sin(\beta)]$$

Substitute $x_1 = R\cos(\alpha), y_1 = R\sin(\alpha)$,

$$x_2 = x_1\cos(\beta) - y_1\sin(\beta)\ \ y_2 = y_1\cos(\beta) + x_1\sin(\beta)$$

Since the change in x, y and heading is going to be small as the frequency of the Coin-D4 is 10 Hertz, we can linearize the functions, by taking their gradients near 0.

$$\cos(\beta) \approx 1, \sin(\beta) \approx \beta$$

$$\therefore x_2 = x_1 - y_1(\beta), y_2 = y_1 + x_1(\beta)$$

Afterwards, we add translation as Tx, Ty

$$x_2 = x_1 - y_1(\beta) + T_x, y_2 = y_1 + x_1(\beta) + T_y$$

$$\therefore x_2 - x_1 = -y_1(\beta) + T_x, y_2 - y_1 = x_1(\beta) + T_y$$

When we put it in a matrix form,

$$\begin{bmatrix} -y_1 & 1 & 0 \\ x_1 & 0 & 1 \end{bmatrix} \begin{bmatrix} \theta \\ T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \end{bmatrix}$$

And we can add as many points as we want to the equation

$$\begin{bmatrix} -y_1 & 1 & 0 \\ x_1 & 0 & 1 \\ \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} \theta \\ T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \\ \vdots \end{bmatrix}$$

To solve this Ax = B, we take the pseudoinverse of A multiplied by B, which we calculate using the numpy library.

Now that we have the transformation matrix, we need to identify which point on the wall does the lidar reading correspond to, in our case we find the shortest perpendicular distance between each wall and the reading.

$$dist = \hat{v} \cdot l,$$

l is the vector of the lidar reading from the start of the wall: if it is a vertical wall, we only solve for the x-value and vice versa for a horizontal wall.

localise_iter(): Since we are dealing with an overdetermined system, there is no exact solution, but only a rough estimate. Running it with more iterations will help minimise the errors.

We tested it with a set of fake data which started with the mini-sets that have far fewer readings, before moving on to set 1 which has 360 points, closer to what we will expect in the actual run. After debugging, we realised that it was able to get close to the actual pose with only one iteration. Additionally, we timed it and it took roughly 1/50th of a second, meaning it is able to keep up with the lidar which takes 10 readings every 1 second.

There were many bug fixes / improvements made to the ICP programme, including:

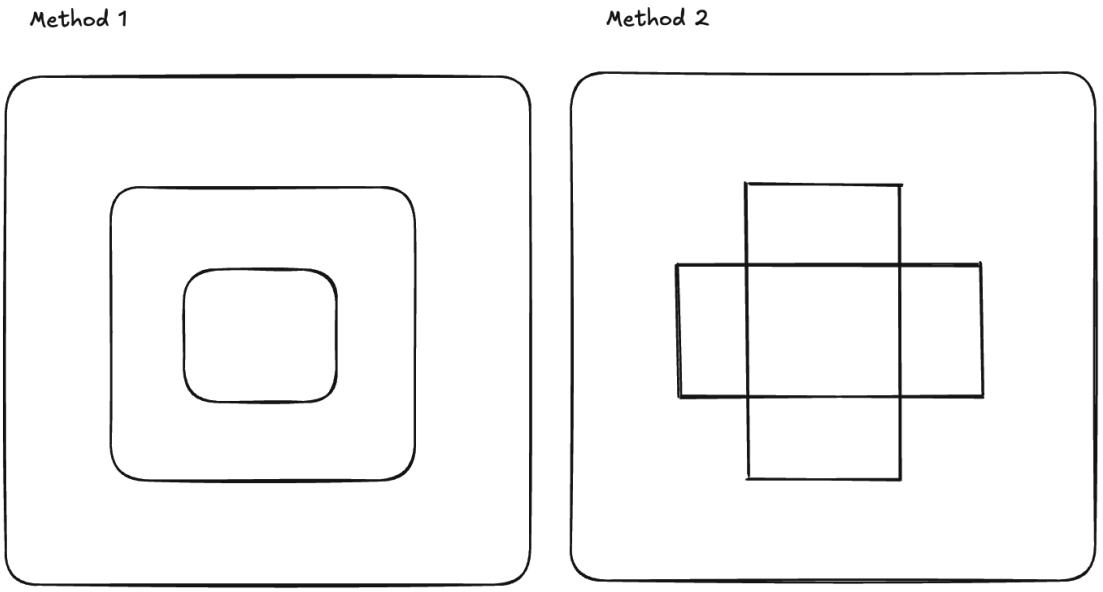
- Omitting the section for rotating the odometry position and only changing the x, y values by Tx and Ty. This would only work if the x, y values of the robot were close to the origin, which is why we observed that the x, y values became very far off when the robot was further away from the 0, 0 point.
- Fixing the vertical wall matching

- Added localise_iter() func to easily adjust the number of times ICP is applied, we found that after 2 iterations or even 1 iteration (if the current pose is not far off from the actual pose), the predicted values were already close to where the function converges

```
[2519.9815226438686, 2782.166225377736, 14.07992247383642] # 1st iter
[2558.977006916372, 2578.8532799108843, 10.352319511258976] # 2nd iter
[2557.2214616516326, 2535.241481238653, 10.300720600786919] # so on and so forth
[2557.2218337089575, 2535.2418511534856, 10.300720660915843]
[2557.221833708957, 2535.241851153486, 10.300720660915838]
[2557.221833708957, 2535.2418511534856, 10.30072066091585]
[2557.221833708957, 2535.2418511534856, 10.300720660915845]
[2557.2218337089575, 2535.241851153485, 10.300720660915859]
[2557.221833708958, 2535.2418511534856, 10.300720660915852]
[2557.221833708957, 2535.2418511534856, 10.300720660915845]

current pose: [2300, 2800, 12]
actual pose: [2557.221145629883, 2535.240325927734, 10.297032712854872],
point_cloud_pose: [2557.221833708957, 2535.2418511534856, 10.300720660915845]
```

- Added parking walls for obstacle challenge
- Added extended inner walls for open challenge: we decided to draw them as nested squares instead of the cross shape on the right, because the cross shape has intersections which may cause some LiDAR points to be matched to the wrong walls. .



Complementary Filter

It combines both the odometry pose and localisation pose by applying a weighing factor to each reading to blend both of their strengths. The odometry pose uses the gyroscope which provides smooth, fast and accurate short-term measurements, but drifts over time; in contrast, the localisation pose is derived using the LiDAR sensor readings which provides long-term reliable readings but are noisy and sensitive to sudden movements.

$\theta_{k+1} = \alpha(\theta_k + \omega \cdot \Delta t) + (1 - \alpha)\theta_{accel}$, where θ_{accel} is instead the angle calculated from the lidar readings. The advantages of using the complementary filter is that it is computationally lightweight compared to Kalman filters and can be easily tuned. Additionally, we wanted to fully understand the theory behind the concepts we were applying by programming our own software which is why we decided not to dive into Kalman filters.

Testing

After testing the robot with only the odometry pose, running it for three rounds, we found that the robot's predicted pose was quite close to its actual pose, with its heading being off by 2 degrees and its x, y values being off by a few centimetres.

With Spike Landmark

POSITION_FILTER_RATIO = 0.1

HEADING_FILTER_RATIO = 0.01

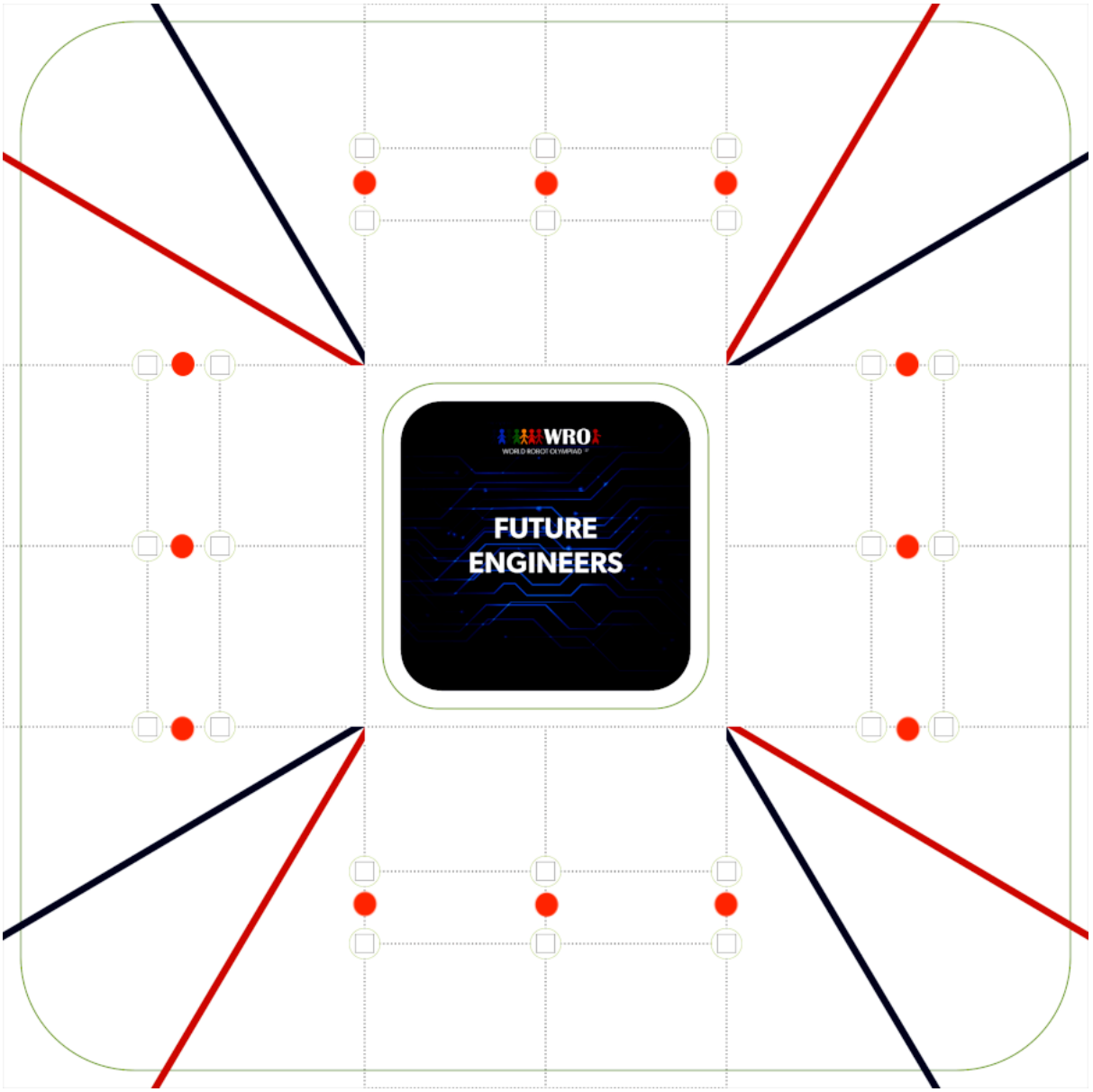
With Iterative Closest Point

POSITION_FILTER_RATIO = 0.1

HEADING_FILTER_RATIO = 0.1

With the spike landmark approach, the weights for the localisation pose are much smaller than with the iterative closest point, because the pose derived from the iterative closest point is much more accurate.

Camera Tracking



- Plotted points along the playfield, in between every pair of squares, where the obstacle could be placed
- Camera will track two points ahead of its current position
- Reused an existing function (dir_to_point) to calculate the angle the servo needs to turn to the obstacle, but adjusting the range to -180 to 180, as the function is used for cases where the range is 0 to 360

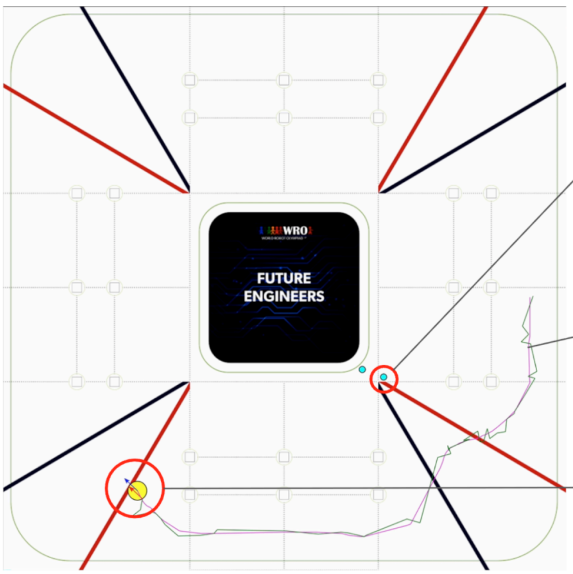
Path following

- drive_path(), drive_paths():
    - Find the error by taking the dot product of the unit vector perpendicular to the path and the vector from the start of the path to the robot (robot_vector): perpendicular displacement $= \hat{u} \cdot \vec{r}$
    - Proportional control: Get the correction / perpendicular distance of the robot from the path by multiplying the error by a gain
    - Add the target_dir of the path with the correction which is what the robot will steer towards
    - Keep checking the dist_travelled_along_path if it reaches the end by taking the dot product of the unit_path_vector and the robot vector: displacement along path $= \hat{p} \cdot \vec{r}$

Debugging / Telemetry tools

To make it easier to visualise the different variables — robot pose, lidar readings, matched spikes, colour of block detected - we created a telemetry software, using sockets to transfer data from the raspberrypi (client) to a computer (server) with our own protocol. We also keep the messages short so that it all fits in one packet and is not sent too frequently. Afterwards, we used tkinter to draw out the points on the playfield with the exact values written out at the left side of the interface.

Additionally, we changed our intialiser.sh file for easier debugging. Now, every time we run, the programme will log the standard output and error messages to a file with the date and time. To change the mode of the robot (sleep, open, obstacle), we will create a softlink between main_obstacle/open/sleep.py to main.py.



Matched spikes (corners of the wall)

Green line: estimated position of robot using spike position

Magenta line: combination of spike position and odometry position

Robot's current position

Hardware Documentation

After a long break from WRO Internationals 2025, we decided to come back to the studio and start work on our next robot for Future Engineers 2026. While our 2025 FE robot was commendable, it had multiple flaws that needed fixing. Upon starting our preparation, we always fix the mechanical issues first, before moving on to improving our code for the robot itself. As such, we came together and formed a list of flaws that our previous robot had.

- **Large robot base** - in turn causing a wide turning radius
    - Led to many near-miss instances of the robot and the obstacles during our obstacle challenge runs in the competition
- **Ackermann Steering Mechanism Issues**
    - **Steering wheel components were loose** - attached only by the pivot point, and nowhere else
        - Because of the singular mounting point, the steering wheel segments often started to bend outwards, due to the force being applied downwards on the base because of our electronic components on the top plate of the robot
        - As the steering wheels bend outwards, steering starts to become less smooth, and turning radius is also slightly affected
        - Even worse, our steering wheels would sometimes come out due to this looseness issue
        - However, this looseness did provide a small amount of suspension for our robot, which allowed it to have more grip on uneven surfaces (at times when the field can get slightly offset)
    - **Servo motor for our steering mechanism was not centred**
        - To adapt to this issue, we initially cut a slot in our steering component for the parts to have more slack when turning, making it smoother
        - However, this slack in the parts also caused the steering mechanism itself to be quite unsturdy and loose, which could sometimes lead to inaccurate turns in our robot runs
    - **Turning radius too small + inaccuracies in the steering geometry**
        - Our Ackermann Steering mechanism was not able to achieve a turning radius sharp enough for our robot to make sharper turns
        - Conceptual misunderstanding of the steering geometry also resulted in inaccuracy in terms of how our mechanism steers the robot

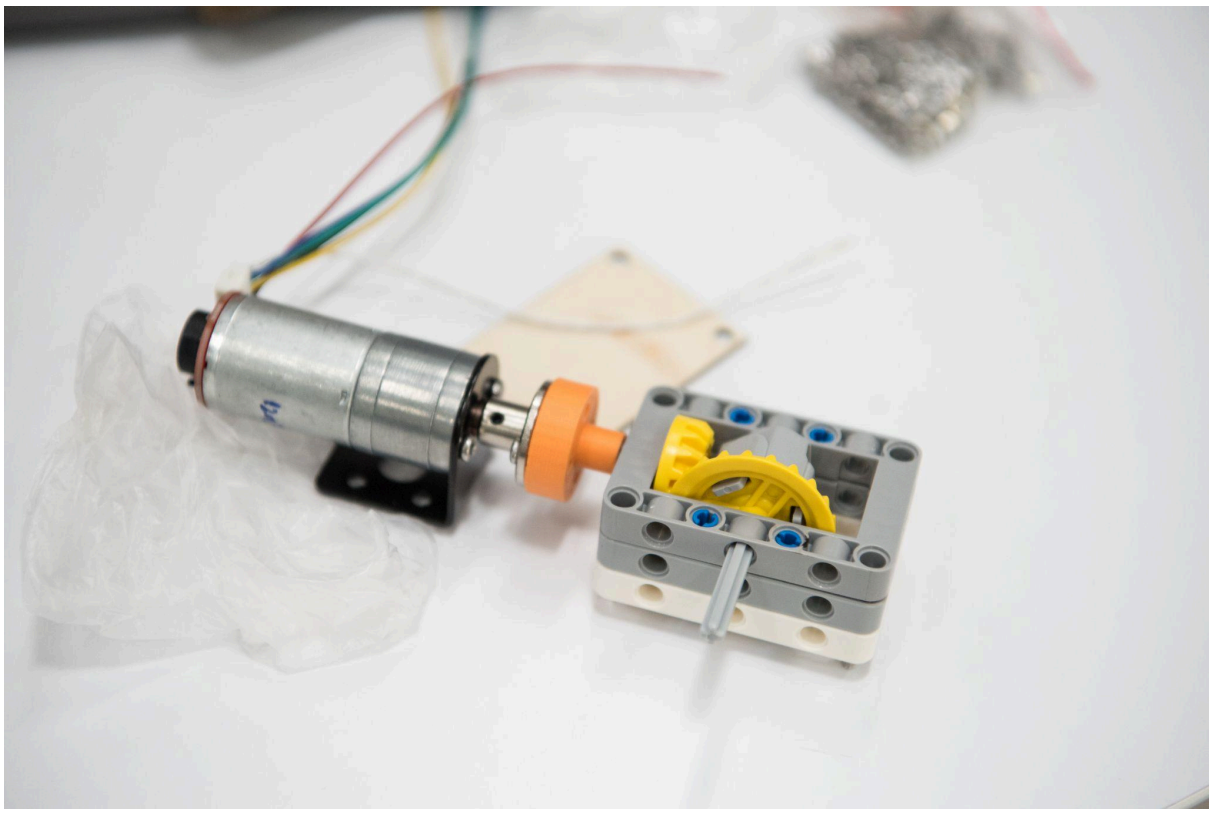
- **Camera was fixed in one direction** - entire robot had to turn to allow the camera to see obstacles on the far end of its POV
    - Despite using a wide angle lens for our camera last year, it was often not wide enough to detect obstacles at the far end, which may sometimes not appear in the camera's POV
    - Results in making incorrect movements to avoid the obstacles, losing points for our competition runs

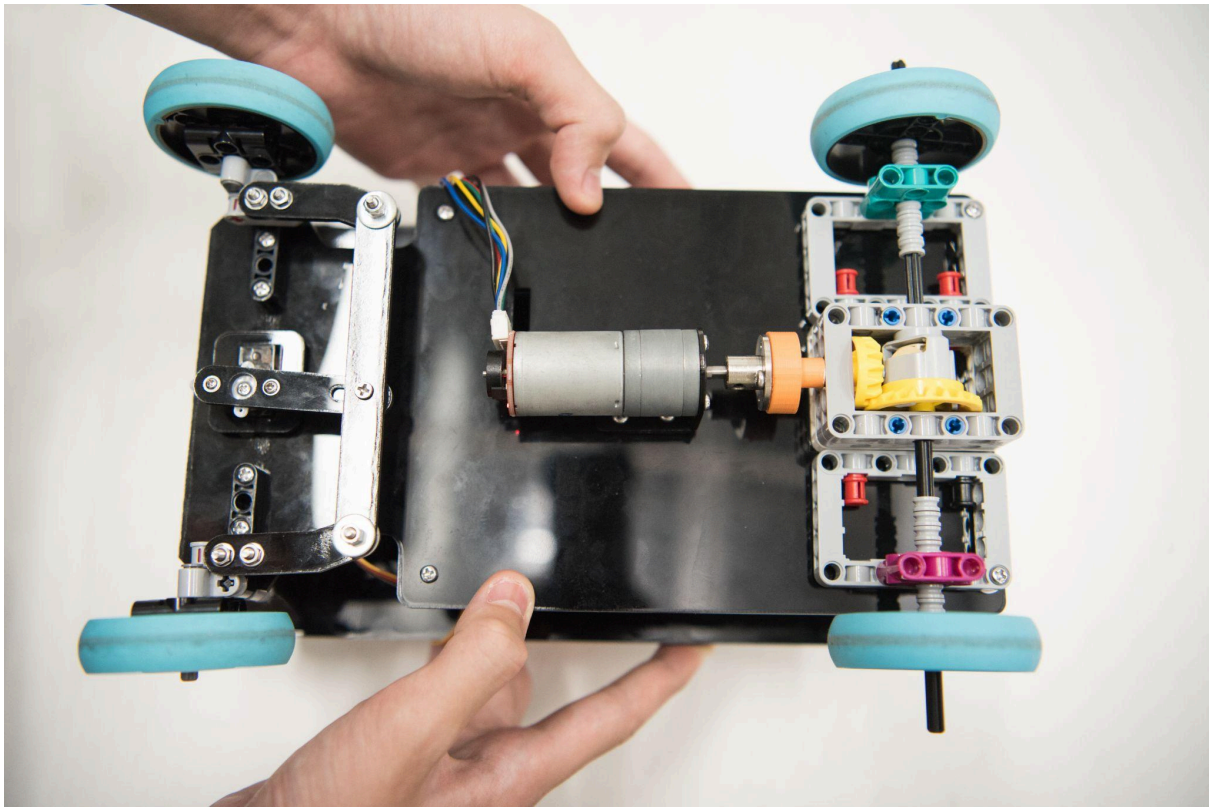
After forming this list of main issues, we decided to tackle these issues one by one, throughout a few studio sessions.

Reducing robot base size

The first issue we fixed was the **large robot base**.

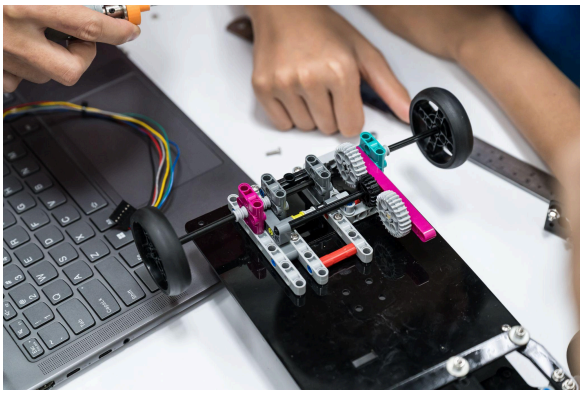
- This problem arguably had the most significant impact on our performance, as it was the lead contributor to our failed runs last year
- Because of this significance, this issue also came with major changes that had to be done to our robot
- This is because simply shrinking our robot frame was not as easy as it seemed; we would need to make our drive base + our Ackermann Steering segment significantly more compact, as well as shrink the layout of our electronic components on the top plate
- This process would last a few sessions, but we mainly focused on shrinking the drive base this session
- Initially, our drive base motor, driving the entire RWD module, was mounted perpendicular to the RWD module itself

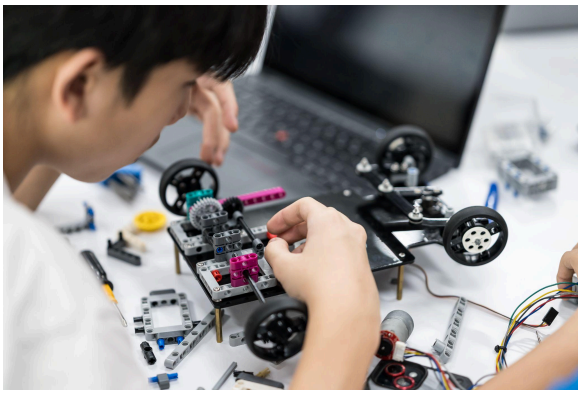




- As seen above, this was the most convenient and direct way to mount our drivebase motor at that time
- However, the DC motor we are using was decently long, and by mounting it perpendicularly this way, it took up more than ⅓ of our previous robot's total length

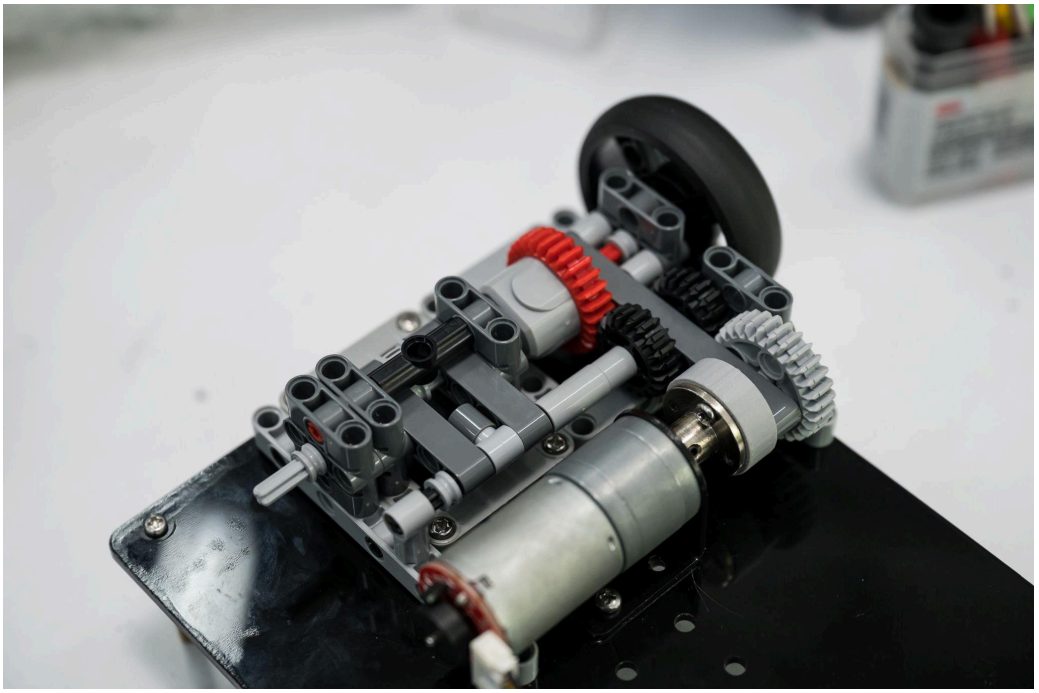
- We realised that if we wanted to make our robot smaller and more compact, we would need to mount this DC motor in another way
- As such, we started to take apart our previous RWD module, and start from scratch again.
- After significantly reducing the size of the RWD module, we still had to similarly reduce the size of our Ackermann Steering module in the front of the base plate, so that we could then reduce the size of the entire base plate
- Initially, our steering module was not designed to be compact, as it was our first ever iteration
- However, in last year's competition, because of how wide our turning radius was, the robot often struggled to avoid the obstacles during its runs
- The solution to this came in two parts; one was to shorten the length of our robot, reducing the distance between our front and back wheels; and the other was to reduce the distance between our steering wheels, to make our steering angle sharper





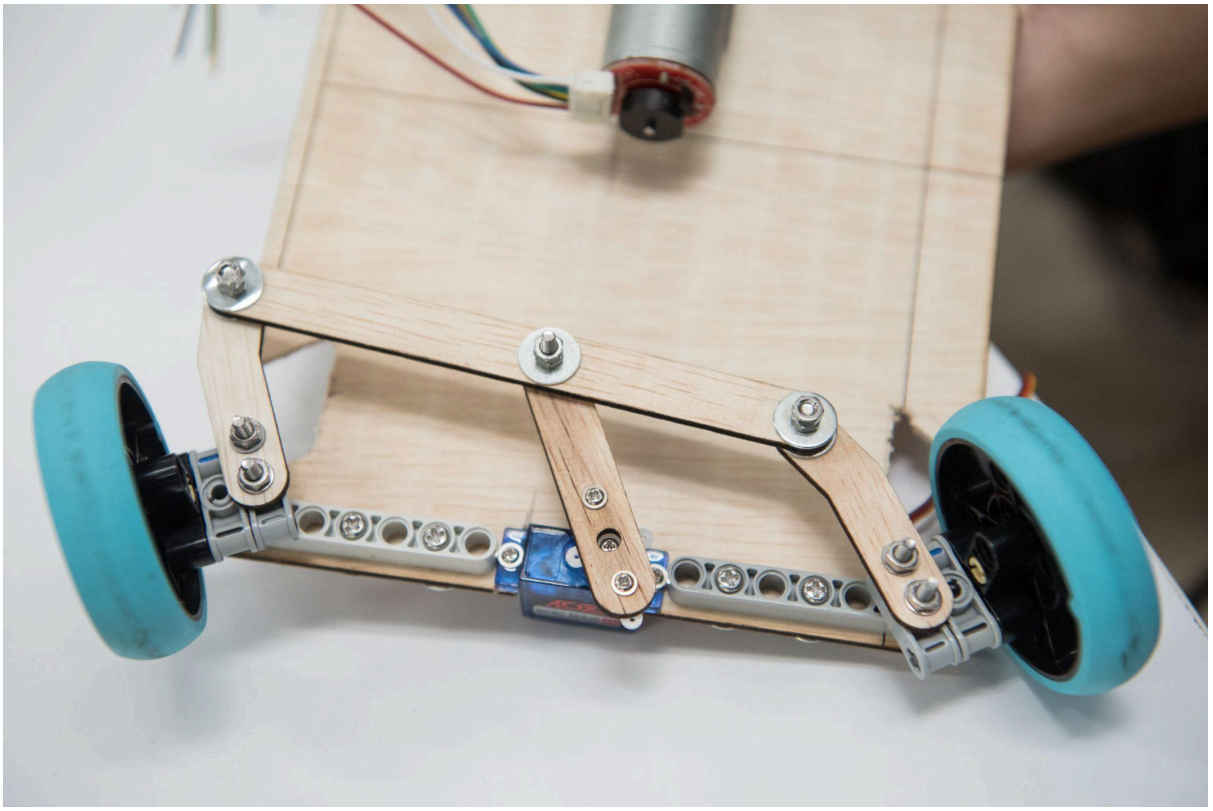
- Over multiple prototypes and a tedious process of trial and error, we eventually built a suitable RWD module that was much more compact than the previous iteration
- This was mainly achieved by mounting our DC motor parallel to the RWD module, instead of perpendicular
- This cut approximately 8-9cm from the total length of our robot base, a significant cut from our previous robot
- However, mounting the DC motor in a parallel orientation came with a few issues, mainly because the motor would either be too close or too far to the rotating differential gear and shaft
- The solution that we eventually found was to gear the DC motor shaft to our main RWD module shaft

- This ensured our motor was mounted in an optimal position, while also slightly increasing the speed of our drive base due to the gear ratio of the shafts
- In addition, there was a lot of useless space taken up by our wheel shafts that could be cut from both sides to reduce the width of our robot
- After all these changes, our new RWD module looked like this:



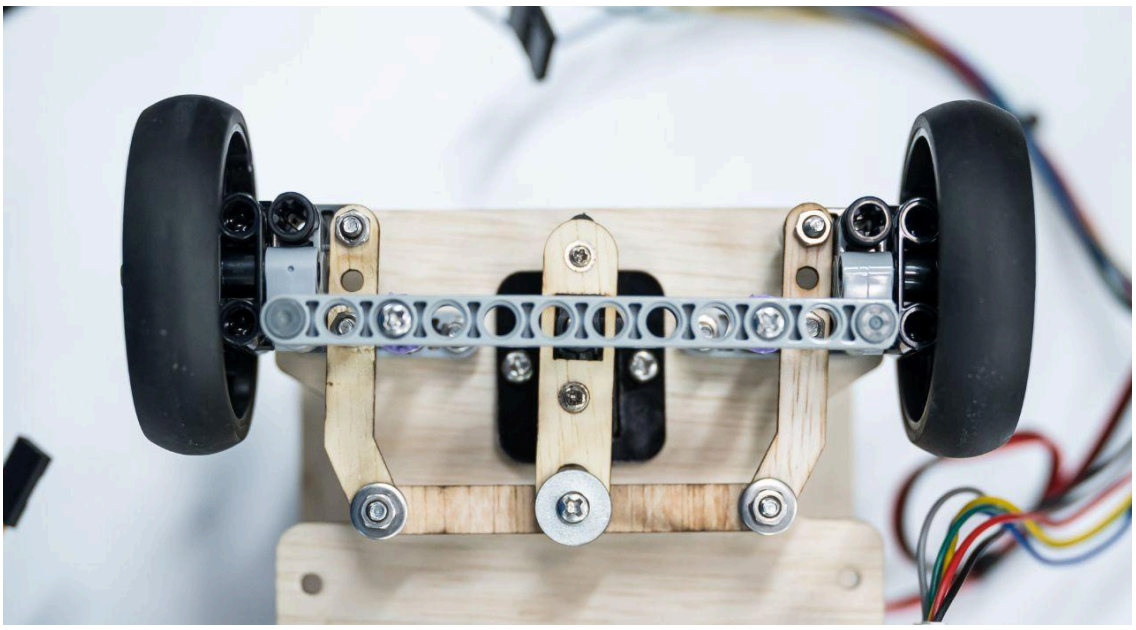
- Ultimately, we not only cut 8cm in length, but also 4-5cm in width.

- After significantly reducing the size of the RWD module, we still had to similarly reduce the size of our **Ackermann Steering module** in the front of the base plate, so that we could then reduce the size of the entire base plate
- Initially, our steering module was not designed to be compact, as it was our first ever iteration



(^^ first steering mech iteration)

- However, in last year's competition, because of how wide our turning radius was, the robot often struggled to avoid the obstacles during its runs
- The solution to this came in two parts; one was to shorten the length of our robot, reducing the distance between our front and back wheels; and the other was to reduce the distance between our steering wheels, to make our steering angle sharper
- As seen in the image above, the distance between both our steering wheels could be reduced by quite a substantial amount, allowing our robot's width to be reduced as a whole
- This fix was pretty straightforward and simple, and after a short period of time, we were able to reduce the size of the steering module

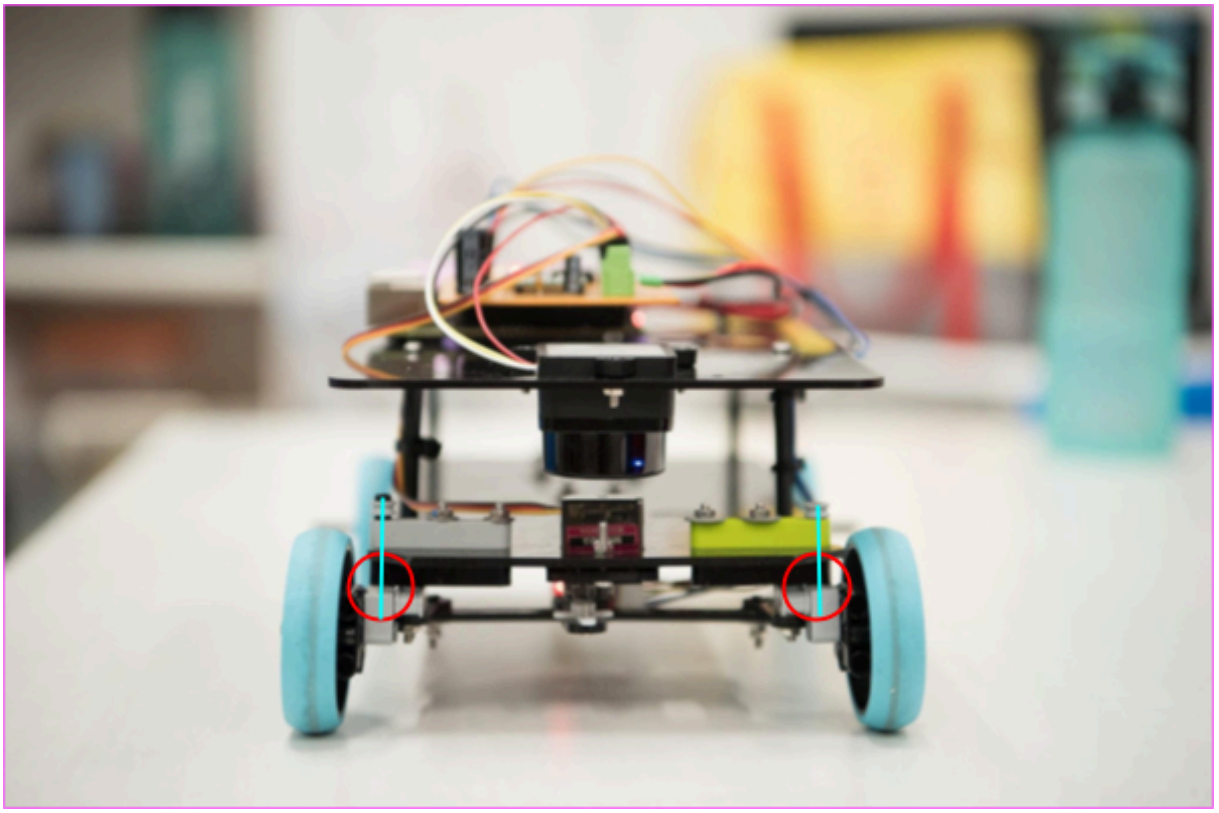


- Comparing both iterations of our Ackermann Steering mechanism, we can see that the distance between both wheels has been shortened from ~18 Lego Technic holes to ~13 Lego Technic holes, which is equivalent to ~40mm, or 4cm
- Because of this change in width, the length of our Ackermann steering liftarms also had to be edited accordingly to account for this difference
- By designing the liftarms online, this process was very fast and we managed to cut out the new liftarms with plywood and connect them quickly

Fixing Ackermann Steering Mechanism Issues

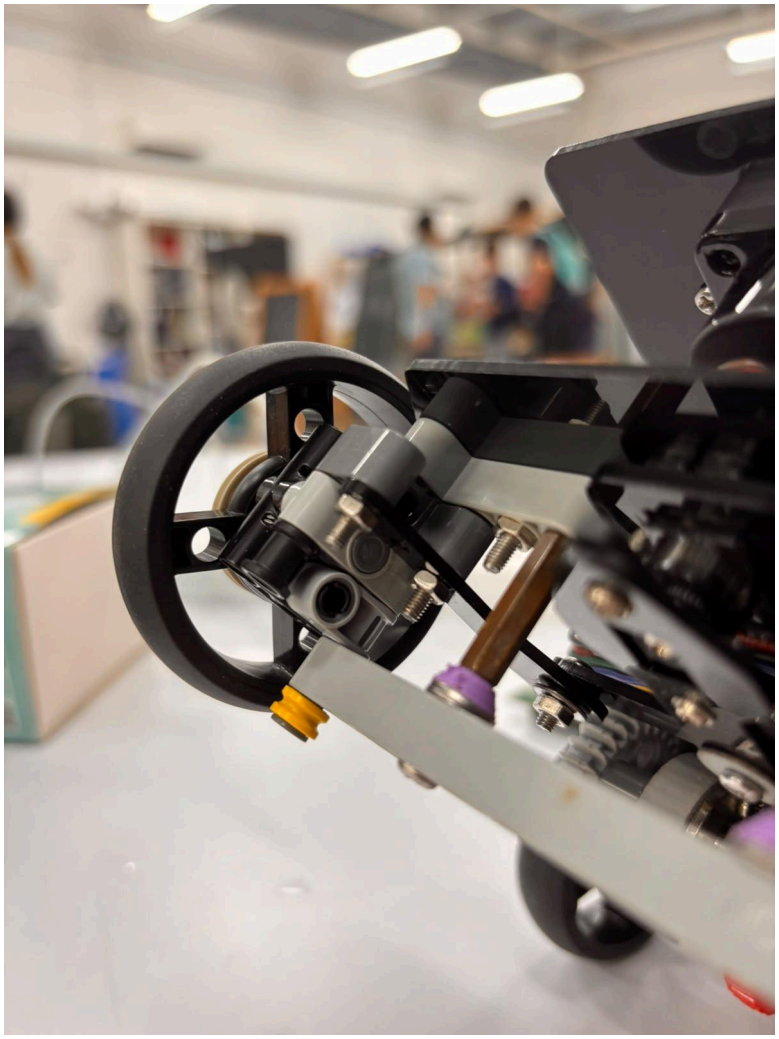
1. Loose steering wheel components

- In our previous robot, the shaft connecting our wheel components to our main robot base plate was short, and only supported by a height of two lego beams
- This meant that the shaft still had leeway to bend slightly

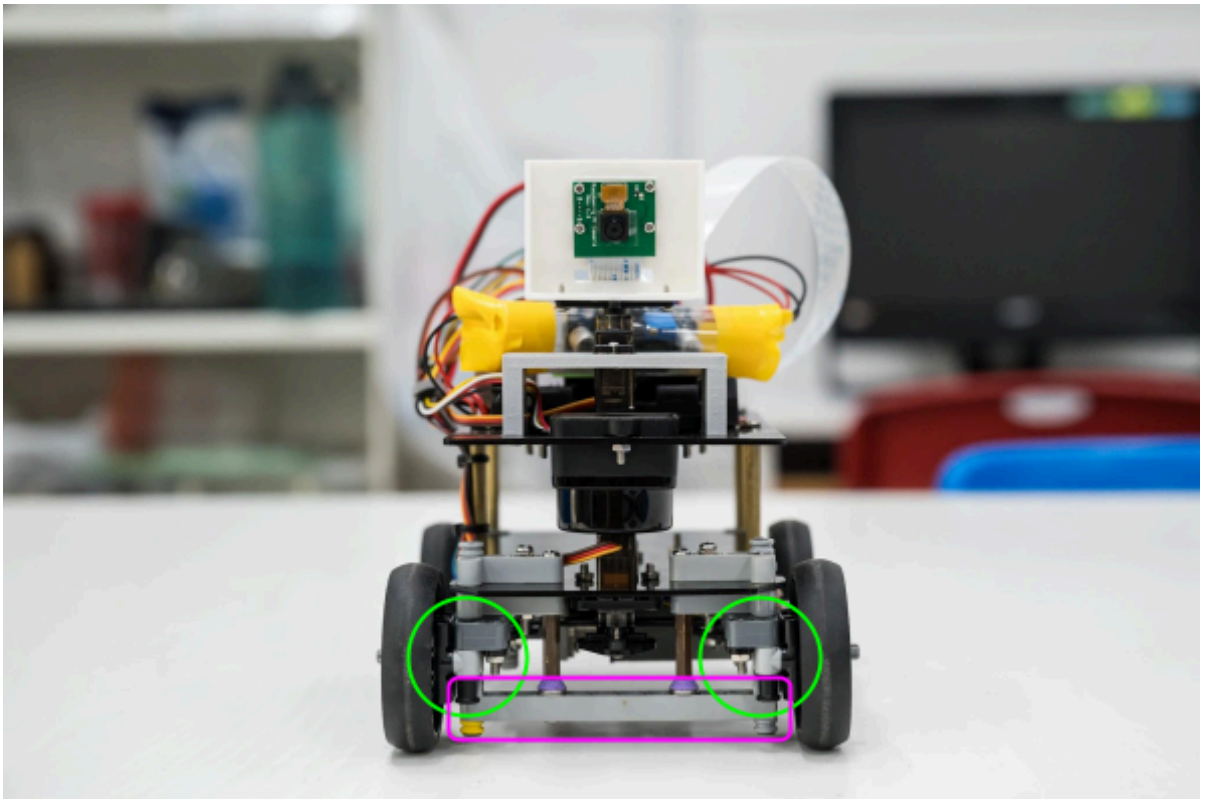


- As seen above, our previous robot is in a stationary state, and the wheels are already bending outwards slightly
- The connection point, indicated by the red circle, is the only connection point for each wheel

Fix #1: We redesigned the component linking the shaft, Ackermann liftarm and rotating wheel together, so that the shaft would be supported by 5 Lego beams compared to 2.



Fix #2: Instead of the wheel components having only one connection point on the top, we added a liftarm below the wheel components, linking both wheels together. This beam pulls both wheel shafts together, ensuring both beams remain parallel to each other, and remain perpendicular to the ground.



- As indicated by the green circles, it shows fix #1, our new and improved wheel module, and the pink rectangle indicates fix #2, the beam that braces both wheels together
- In addition, we braced the beam to the main robot baseplate using two short standoffs, which doubled up as a limiter for our Ackermann Steering linkage mechanism, preventing this mechanism from overcentering
- Overlocking occurs when both Ackermann liftarms form a straight line with the pivot point, and when the servo motor tries to turn back, the force is exerted straight through the joints instead of turning them, since there is zero leverage to start the rotation
- By using these standoffs, it prevents the liftarms from aligning to a straight line, and maintains leverage for continuous rotation and steering

2. Centering Steering Servo

- Another issue with our previous robot was the slight inaccuracy in design in our first iteration of our Ackermann Steering module
- In our previous robot, we mounted the steering servo motor, by centering the rectangular shape of the servo on the robot base plate
- However, this was an issue, as the actual point of rotation is slightly offset from the centre of the rectangular servo motor

- As a result, we had to cut a long slot on our Ackermann liftarm to account for this slight inaccuracy, in turn making our steering looser than usual

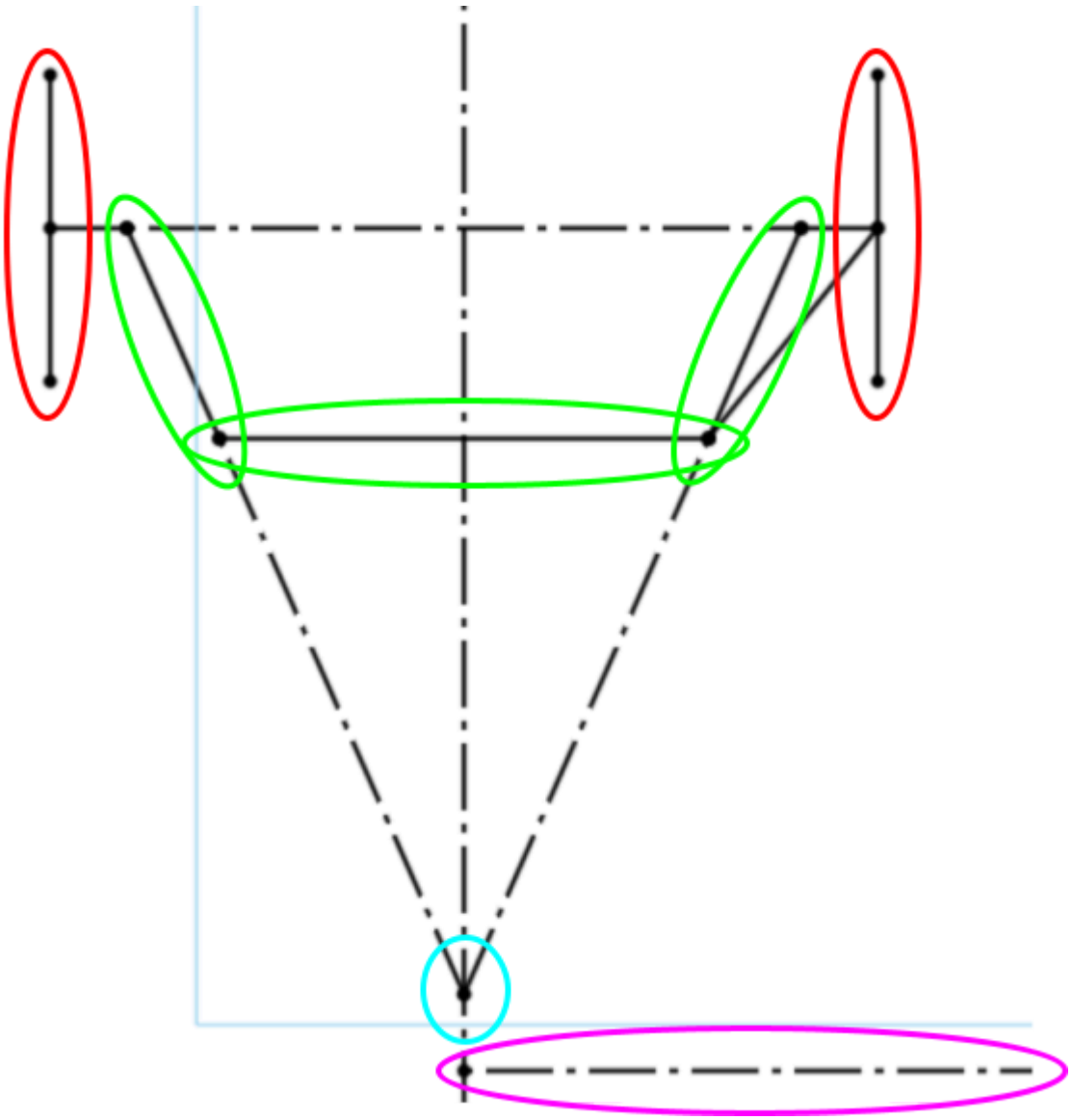


- To fix this issue, we had two options:
    - The first option was to mount this servo motor in the same orientation, but offset it slightly to the left to account for the offset of the centre of rotation
    - The second option was to rotate this servo motor 90 degrees, so the short side of the motor faces the front of the robot
        - In this way, the centre of rotation does remain centred, while the entire servo motor is also centred to the robot
    - The former option, however, did not work after testing, as the servo motor would hit the Ackermann Steering bracing on either side of the robot, and thus would not fit if the orientation was kept the same
- As such, we decided to go with the latter option:
    - This fix was simple enough, as we just needed to rotate the rectangular slot for the servo motor on our robot base plate CAD file
    - Soon after, we adjusted our Ackermann steering liftarms accordingly, and our Ackermann Steering module was complete

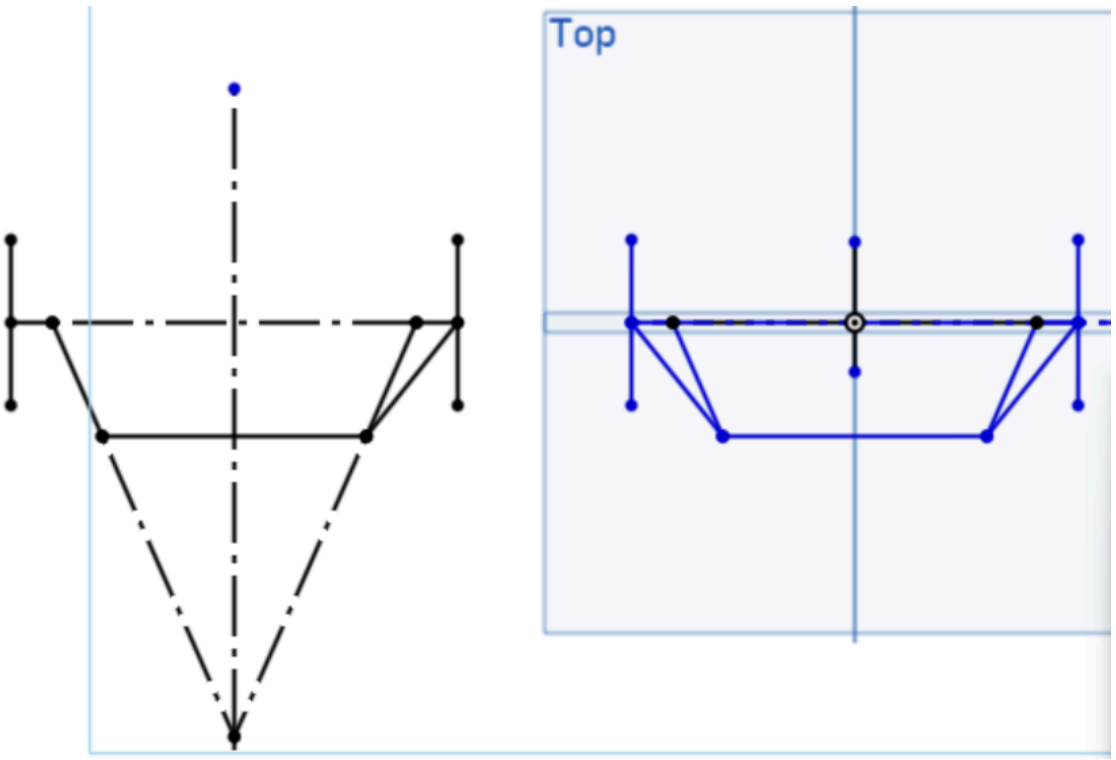
3. Turning radius too small + inaccuracies in the steering geometry

- The third issue, and arguably our biggest lapse in conceptual understanding, was our misunderstanding of how Ackermann Steering is truly achieved
- Last year, we learned about the concept of Ackermann Steering and how a series of linkages would be able to achieve such a concept
- We then put together a simple linkage mechanism, and upon testing, seeing that both steering wheels moved by different amounts, we immediately assumed that this mechanism had so-called achieved “Ackermann Steering”
- This mechanism was thus carried over to this year, where we used the same series of linkages, apart from reducing the overall size of the mechanism
- However, we eventually came to realise that our mechanism did not actually emulate the concept of Ackermann Steering - far from it actually
- In fact, the concept of Ackermann Steering ironically is typically not applied to vehicles nowadays
- In addition, Ackermann Steering was much more complex in nature, and is actually impossible to recreate through linkages alone
- This came not only as an important lesson to not make bold assumptions during testing, but also prompted us to rethink our Ackermann Steering mechanism from scratch
- Since achieving the true concept of Ackermann Steering was a near impossible feat for us in a short amount of time, the next best solution for us was to recreate our series of linkages through a simulator, allowing us to visualise how our existing series of linkages actually steers, as well as figure out how to reduce our turning radius to make sharper turns

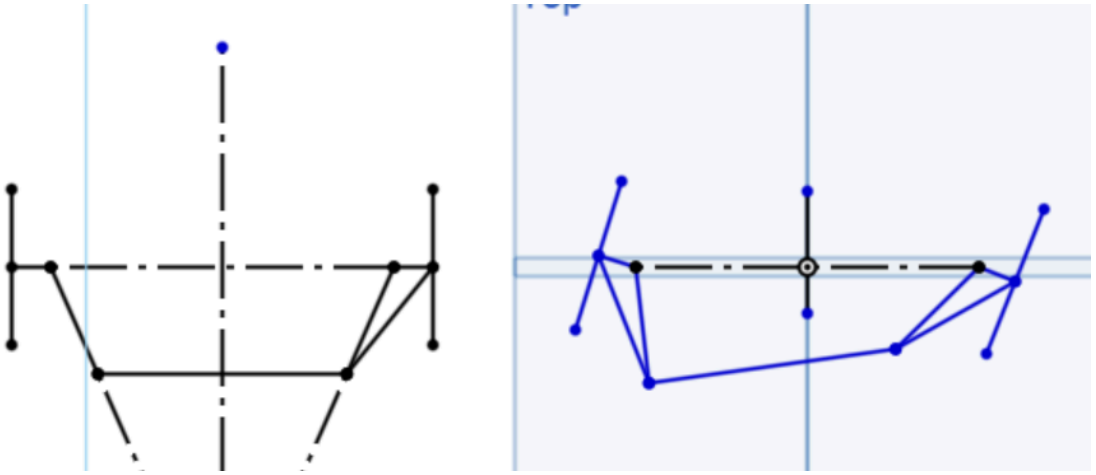
Firstly, we recreated our Ackermann Steering mechanism through Onshape, using a simple 2D sketch with suitable constraints.



- Where the horizontal lines indicated in red are our steering wheels, the lines indicated in green are our steering linkages, the dot indicated in blue is the point at which the lines extended from both steering linkages intersect on the centre line, and the horizontal line indicated in purple is the line extended to the right from the centre of our rear wheels (RWD)
- Appropriate dimensions are set accordingly, to recreate our actual steering mechanism as realistically as possible
- Following this, we duplicated this drawing into the same sketch, and set the respective linkages to be of equal length
- In this way, by slightly changing the dimensions of the first sketch, the dimensions of the second sketch would also change accordingly

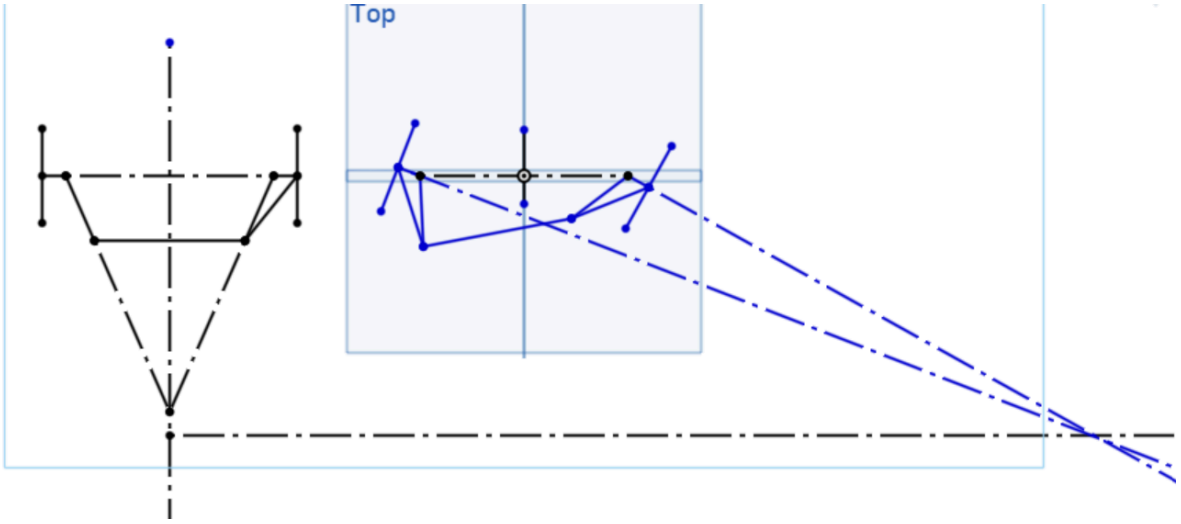


As shown in the above picture, the dimensions of the linkages are all of equal length. In addition, the sketch on the right is indicated in blue, since certain constraints were removed to allow the linkages to move.

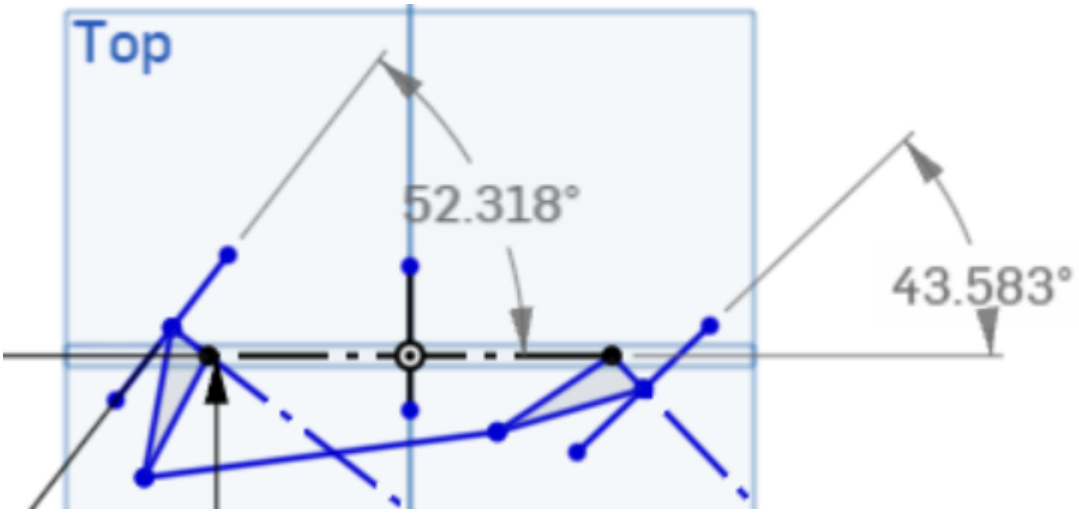


- In other words, the left sketch is fully constrained, where all dimensions are set to predetermine the lengths and angles of the linkages
- These dimensions are then carried over to the right sketch, where the linkages are free to move
- This allows us to, firstly, make dimensional changes to the left sketch, then manually move the linkages on the right sketch to see how the steering of both wheels would be affected

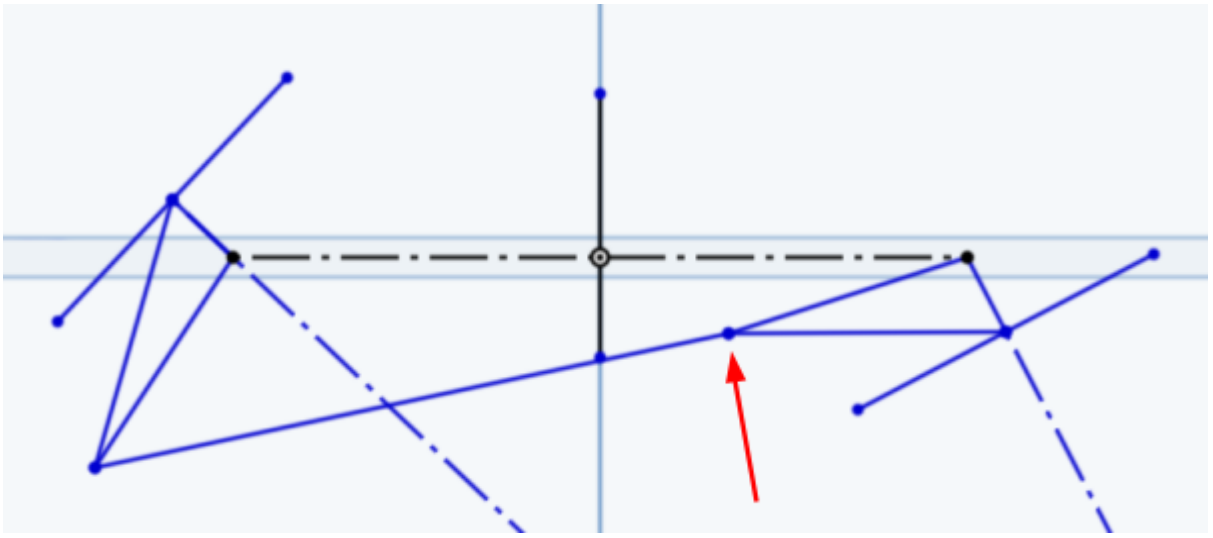
- Through this setup, we were able to experiment and see how the angle of the steering wheel linkages would affect our maximum steering angle as well as how well the linkages emulate the concept of Ackermann Steering
- When making a turn, Ackermann Steering is achieved when the line extended from the steering linkages of both steering wheels always intersect at the line extended from the centre of the rear wheels, at any point in the turn, as shown below:



- By adjusting the point at which the lines extended from the steering linkages intersect at the centre line, which essentially adjusts the angle of the steering linkage, we could experiment to see how it would affect the turning radius of our robot, by repeatedly recording the angles of the steering wheels at the maximum steering angle.

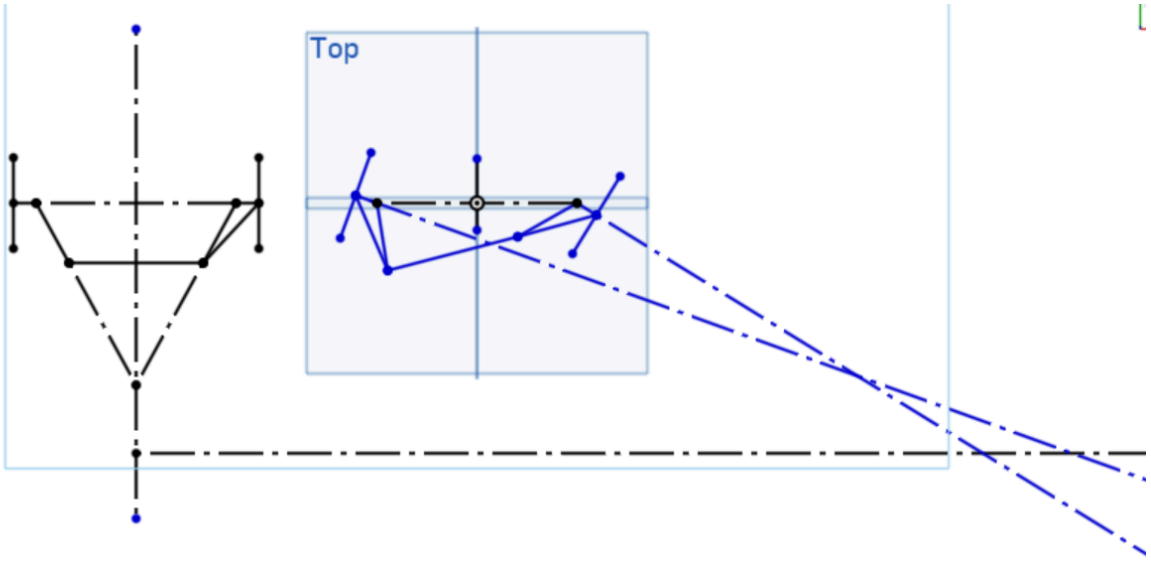


The maximum steering angle is determined by moving the series of linkages to the point right before the mechanism over-centres.



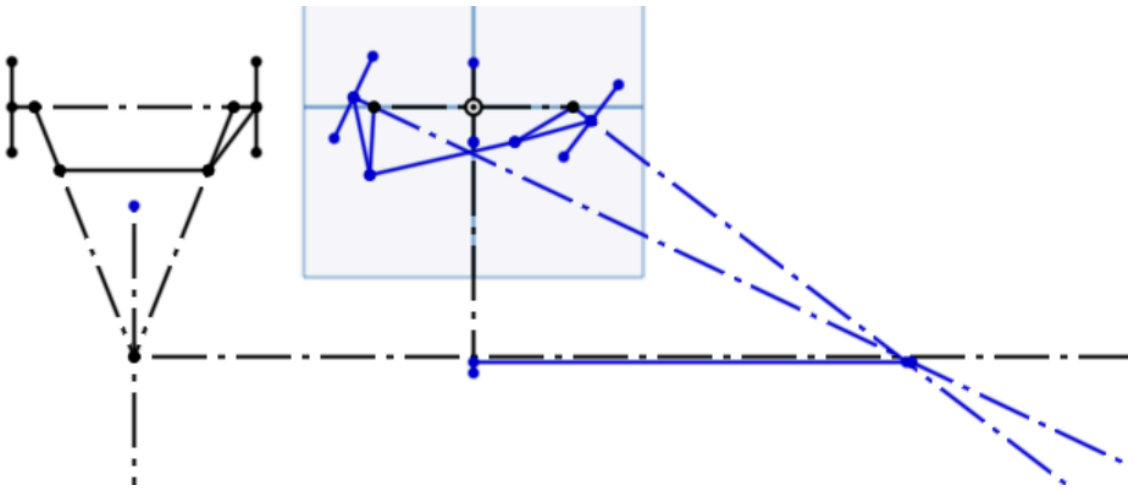
- The image above shows an example of an instance where the steering mechanism would overcentre
- As a result, when the mechanism tries to steer in the opposite direction after this, the mechanism would lock in place and would be unable to steer in the other direction
- As such, all measures of our maximum steering angle is based on the point of steering right before the mechanism would over centre

Below are some of our findings:



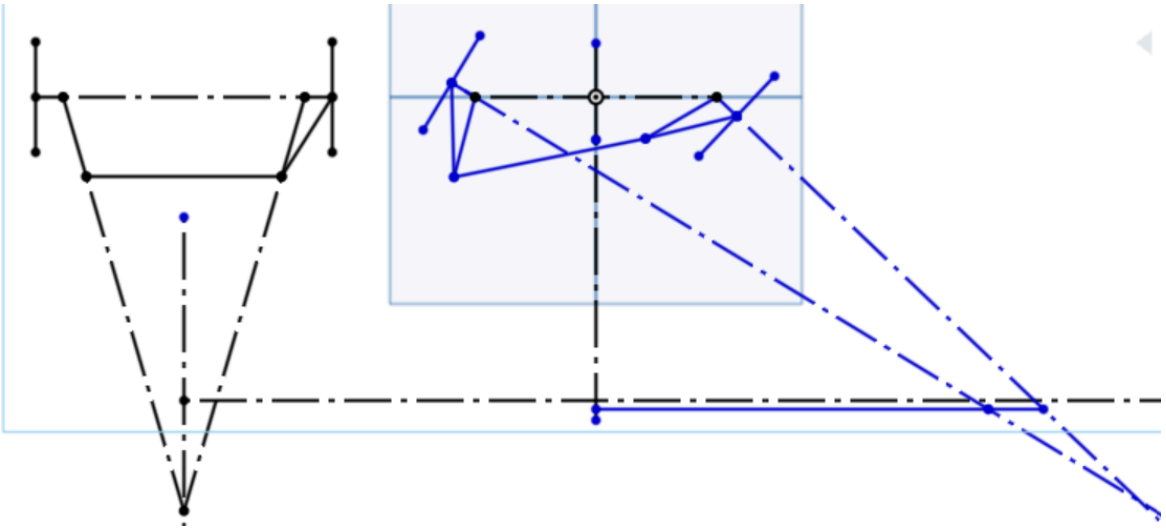
Distance between centre of steering wheels and point of intersection: 80mm

Approximate turning radius: 273~293mm



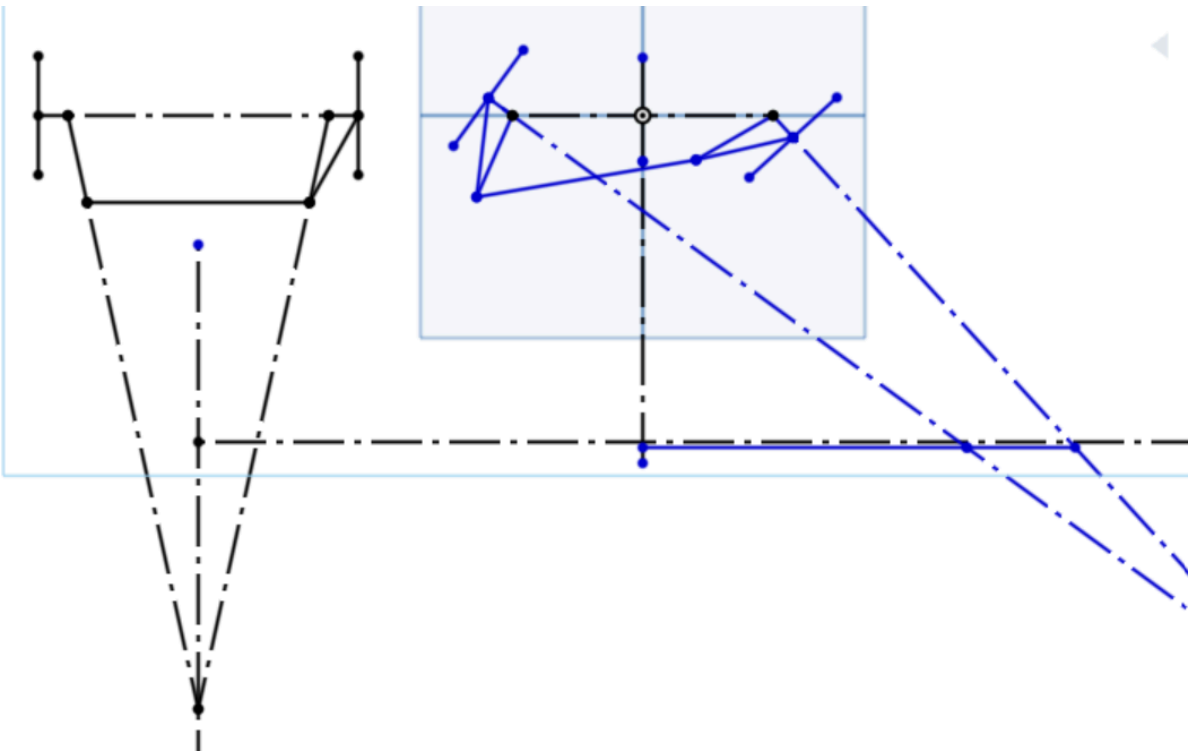
Distance between centre of steering wheels and point of intersection: 110mm

Approximate turning radius: 191~193mm



Distance between centre of steering wheels and point of intersection: 150mm

Approximate turning radius: 142~152mm

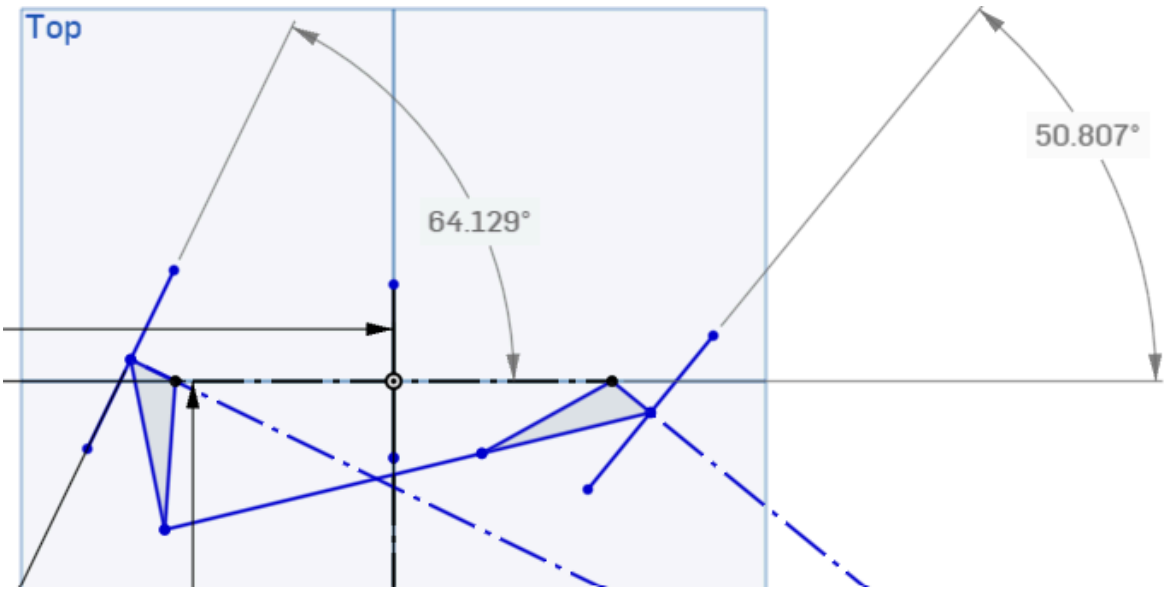


Distance between centre of steering wheels and point of intersection: 200mm

Approximate turning radius: 110~128mm

From the results collected above, the trend observed is that as the distance between the centre of the steering wheels and the point of intersection of the extended lines increases, the approximate maximum turning radius of our robot decreases. Put simply, this meant if we increased the angle of our steering liftarms, our robot would be able to make even sharper turns than before. This is further supported by the difference in the angles of the steering wheels at the maximum turning point, as shown below:

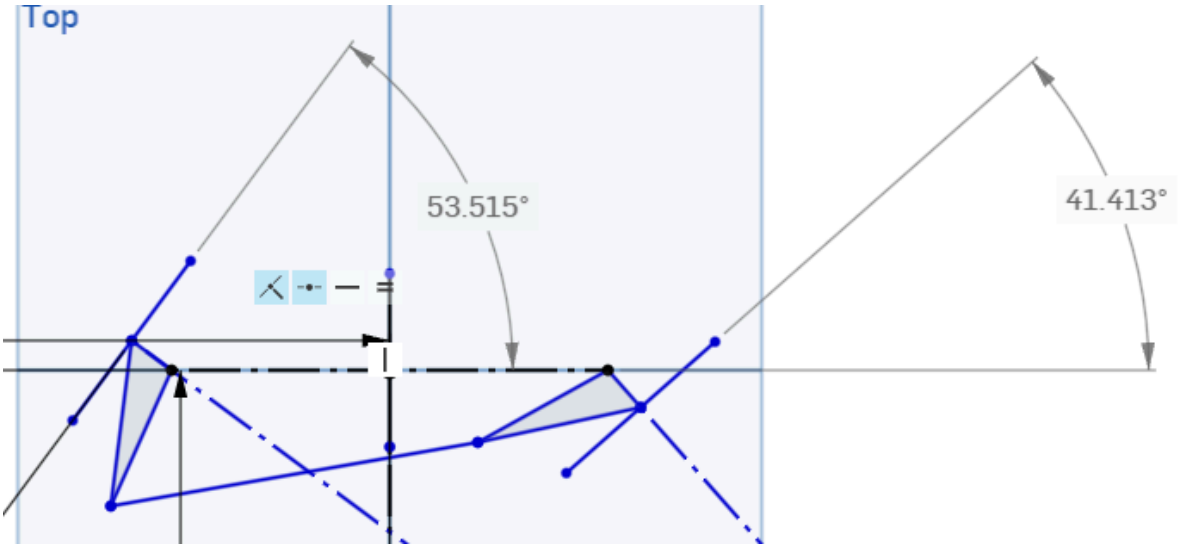
BEFORE - Larger angle of steering liftarm:



Maximum steering angle of left wheel: ~25.9 degrees

Maximum steering angle of right wheel: ~39.2 degrees

AFTER - Smaller angle of steering liftarm:



Maximum steering angle of left wheel: ~36.5 degrees

Maximum steering angle of right wheel: ~ 48.6 degrees

As seen above, by merely adjusting the angle of our steering liftarms to be smaller, we could increase our average maximum steering angle by approximately 10 degrees.

Following this experiment, we then edited our steering linkage liftarms to decrease the angle accordingly. We then proceeded to laser cut these new liftarms, and replaced our old steering mechanism with these new liftarms. Through this, we were able to have a robot capable of

performing sharper turns, allowing for many improvements in efficiency, such as getting out of the parking zone in a reduced number of turns, and parallel parking in a significantly reduced number of precise manoeuvres.

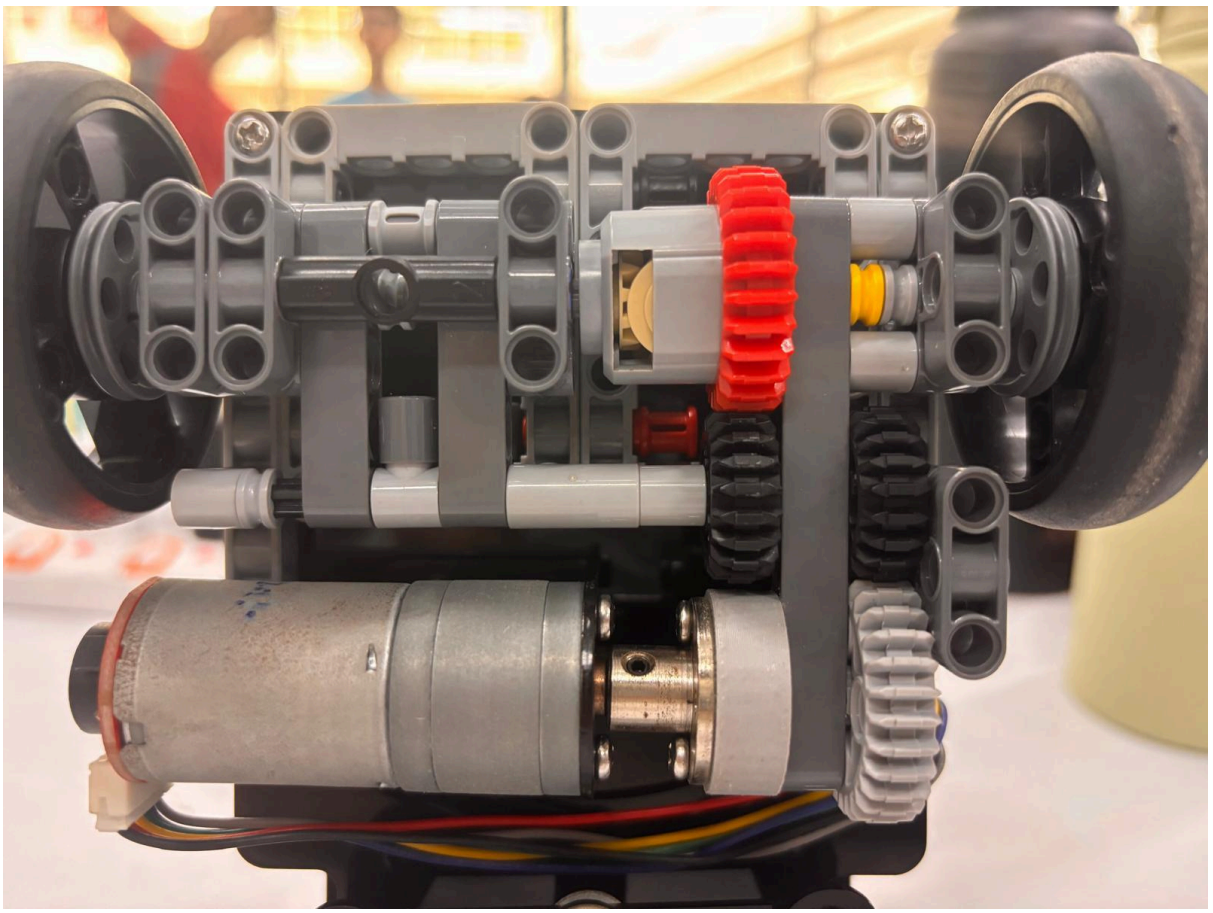
Other Design considerations

1. Base speed

In the previous year's competition, our robot runs were painfully slow, mainly because our Rear Wheel Drive was geared downwards. This meant that even when our RWD DC motor was running at full speed, our robot still moved very slowly. As such, we wanted to increase the speed of our robot this year.

The first and most obvious solution that came to mind was to replace the old DC motor with a new, faster DC motor which could drive our robot much faster. However, after careful consideration and deliberation, we came to understand that using a faster motor may cause a negative impact on the rest of our robot run. In particular, in the way that our robot speed affects our LiDar readings.

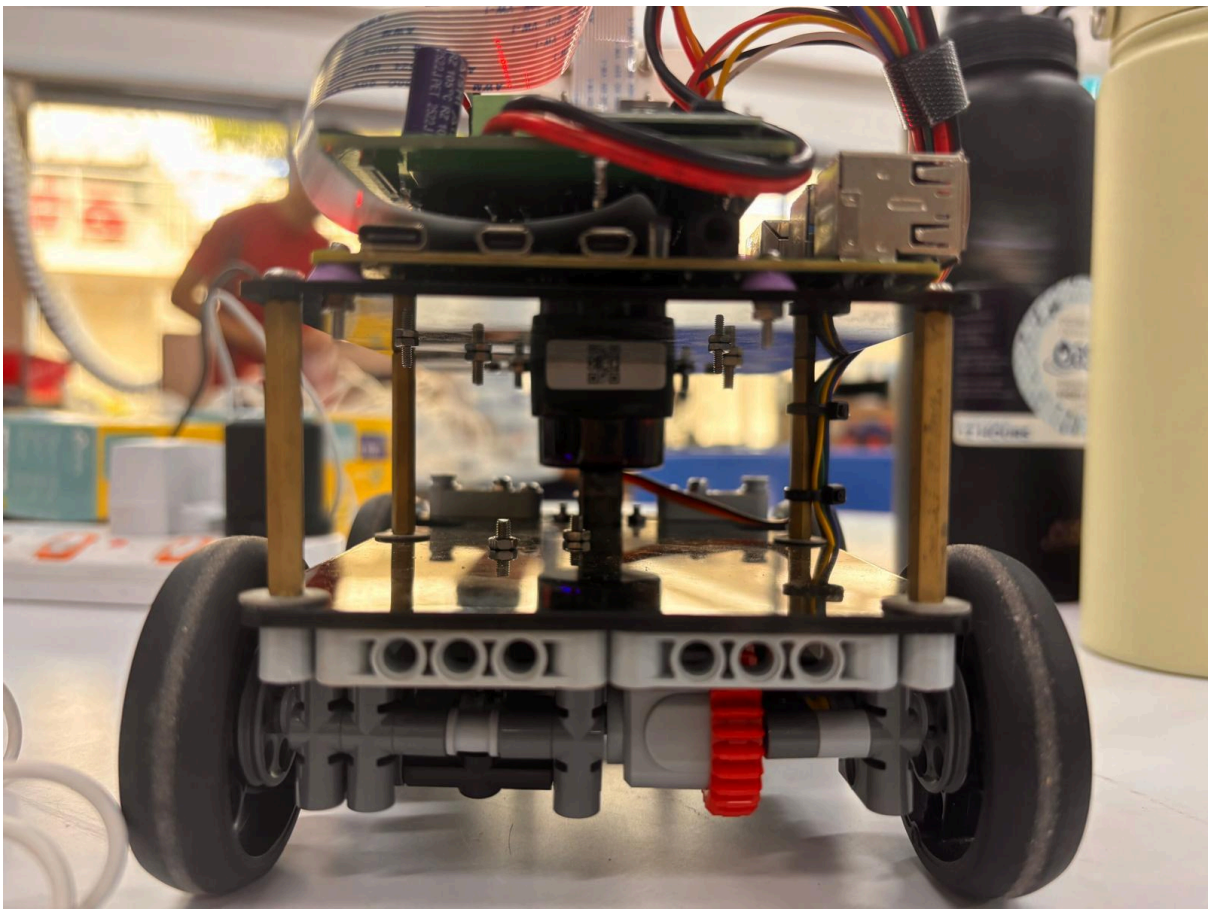
- The Coin-D4 LiDar sensor we are using features a 10Hz scanning frequency
- Take a **slow robot** for example; in 1 second, the robot would move a relatively short distance, meaning that the space between the LiDar readings would be relatively smaller
- This essentially reduces the significance of the impact of the robot's movement on the accuracy of our LiDar readings
- Now, imagine a **faster robot**; in 1 second, the robot would move a much greater distance, meaning that the space between the LiDar readings would be much larger
- This would in turn result in our LiDar readings getting heavily affected and thus inaccurate when moving at higher speeds
- As such, we decided to scrap the idea of using a faster motor, and decided to prioritise consistency over efficiency



However, it is worth noting that through the process of reducing the size of our RWD module, our new and current RWD module is no longer geared downwards, as compared to last year's RWD module, meaning that our robot's maximum speed is actually already greater than before. This meant that our robot runs slightly faster than before, despite not changing the motor at all.

2. Layered design

In the previous year's competition, we decided on a layered type of design for our robot, featuring different layers each consisting of different types of components. This concept was carried over to this year's robot design.



As shown above, the layered design of our robot allowed us to neatly segregate the respective components according to the layers, **as well as** allow for a large, empty space in between both layers, where our LiDar is mounted. This open space allows the LiDar to collect accurate readings from almost all angles, preventing any LiDar readings from being affected by internal robot components which may block the lasers emitted by the LiDar. However, it is worth noting that the open space we designed for the LiDar was minimised, not omitted.

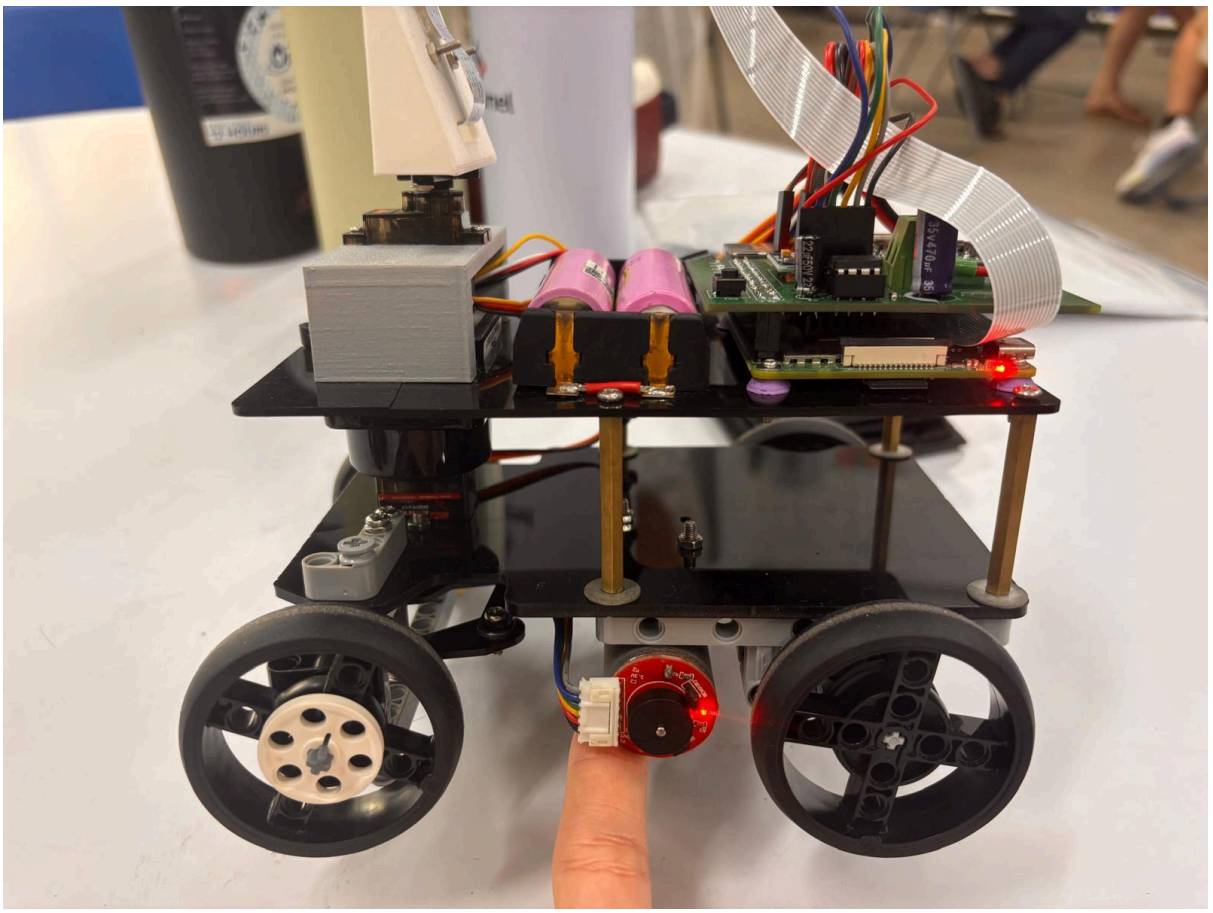
- Since our robot requires at least 4 standoffs to secure both layers together and ensure structural stability, these standoffs were the only components that were impossible to remove
- As such, we mounted these 4 standoffs in strategic locations on the plate, where the angle at which the LiDar would be blocked by these standoffs is not a typically used angle for our measurements
- (For example, we did not mount our standoffs at the front left and right corners of the robot, since that would block the immediate left and right measurements of our LiDar, both of which are critical for our Localisation on the field)

3. Weight balance

On our robot, there are multiple components, some of which are particular heavier than others. Some of the heaviest components on our robot include the Main Raspberry Pi module and the battery holder. The mounting of these components also came with careful considerations regarding the weight balance of our robot.

- If we were to mount the heavier components on the front of the robot, this would increase the force applied on the front steering wheels of the robot
    - This meant that when making turns, our steering wheels would have a smaller chance of skidding
    - However, this would mean that there is less weight concentrated at the rear of the robot, meaning that less force is applied on the Rear Wheel Drive wheels
    - This would mean that there is a higher chance of our rear wheels skidding when the robot is moving
- If we were to mount the heavier components on the back of the robot, this would increase the force applied on the rear, driving wheels of the robot
    - This meant that when moving in a straight direction, the likelihood of our rear wheels slipping is reduced
    - However, this would in turn reduce the weight concentrated at the front of the robot, meaning that less force is applied onto the front steering wheels
    - As such, when making turns, the steering wheels have a greater likelihood of skidding

As such, the best solution for the design of our robot was to balance the weight distribution as much as possible.

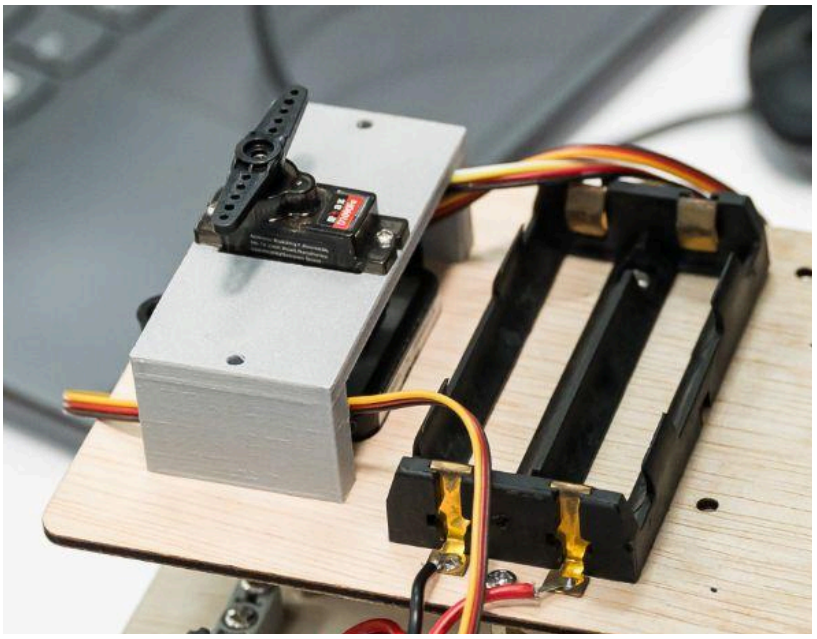


We achieved this by mounting our main Raspberry Pi module at the rear, our battery holder roughly in the centre, and our rotating camera bracket near the front of the robot. Our weight distribution would be even better if our battery holder was mounted near the front of the robot, but since our rotating Camera had to be mounted near the front in order to properly detect the coloured obstacles, the next best layout of our components was for our battery holder to be mounted in the centre instead.

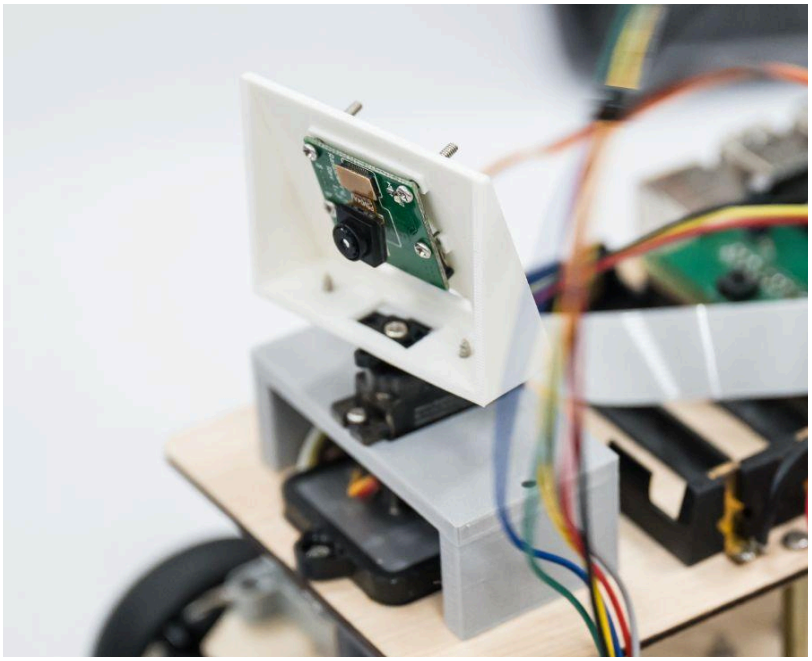
Increasing our Camera range

The last main hardware issue we found in our previous robot was the intermittent blindspots of our camera. Despite using a wide-angle lens for our camera, our robot always tends to miss obstacles at the very edge of the Camera's POV, especially at junctions requiring sharp turns. As such, we decided to make a significant mechanical change to how our camera was mounted. Instead of just a fixed and stationery mount, we decided to attach our camera to a servo motor, with a functionality similar to that of a Camera Gimbal module. To achieve this functionality, we first had to find a way to mount

the servo motor on the top plate of our robot, while not interfering with the other electronic components.

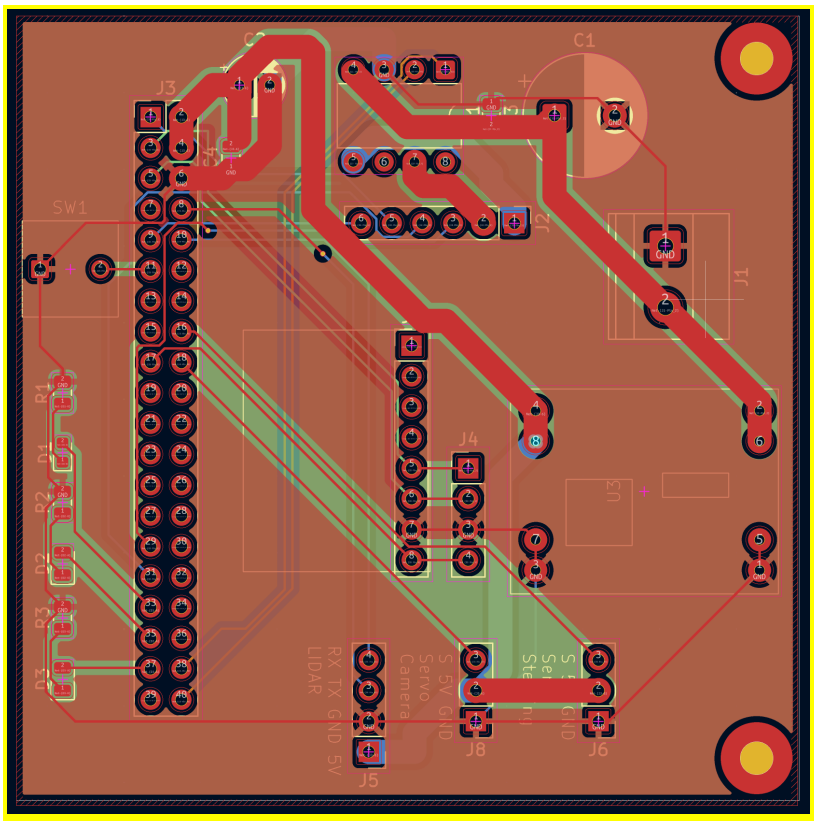


- As seen above, the most convenient location we found for our servo motor was to mount it on top of our lidar sensor
- This was achieved by designing a custom 3D printed elevated bracket, fastened to the top plate, that allows our servo motor to fit perfectly on top
- Thereafter, we quickly attached the camera bracket together with the camera itself, and the rotating camera module was complete

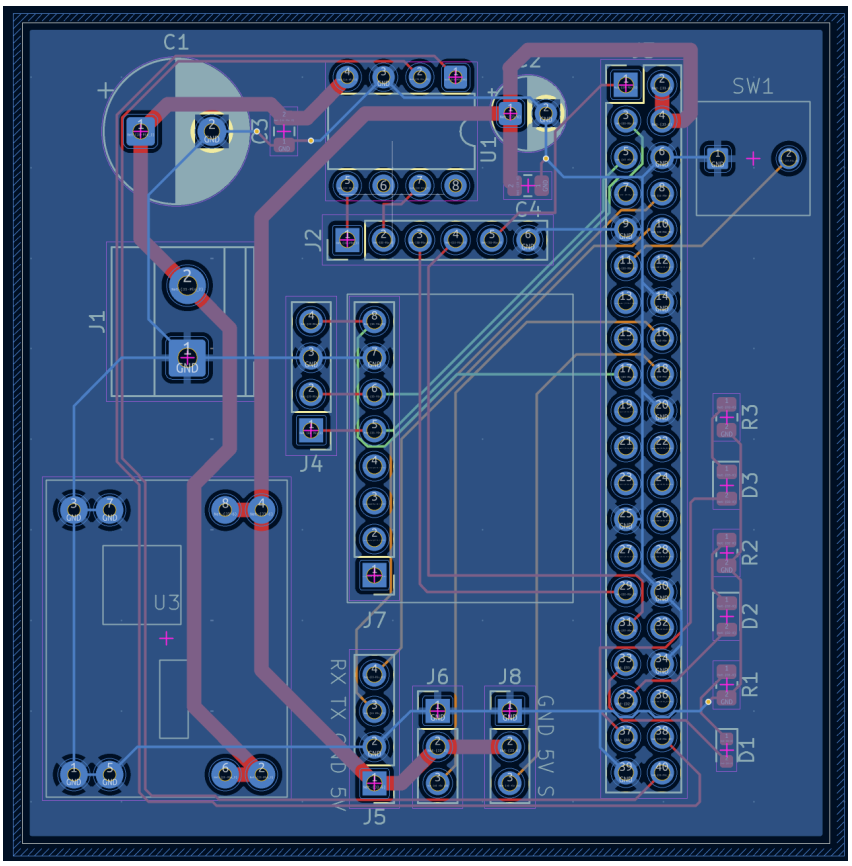


PCB

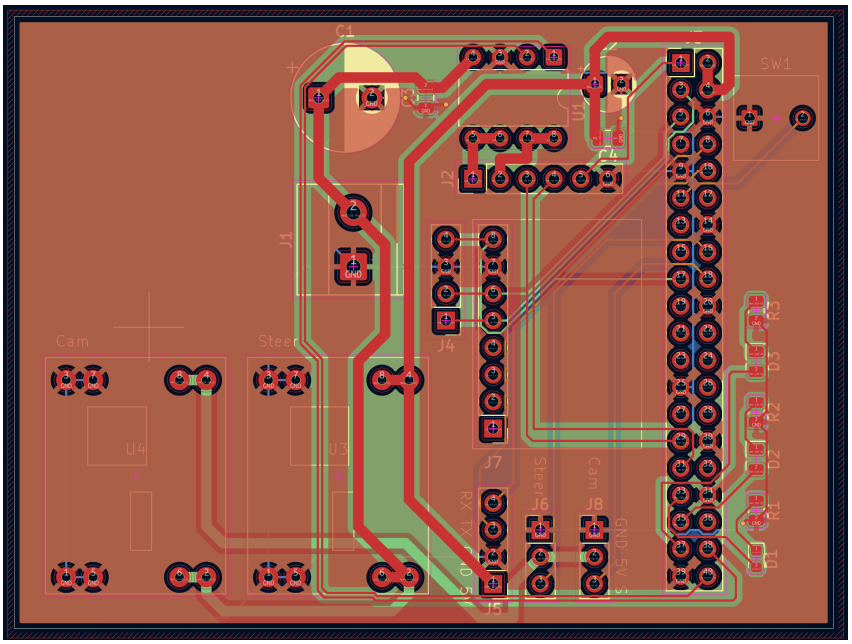
We also redesigned the PCB such that it fits on the smaller top plate without interfering with the battery holder before sending it out for printing through JLCPCB



We only realised after inserting the PCB onto the raspberrypi that the 2x20 connectors were in the wrong orientation, which almost short circuited the raspberrypi, so we had to redesign the PCB board.



While testing with a new servo, we realised that both servos draw a significant amount of energy, which caused the raspberry pi to reboot. Our solution was to add an extra voltage regulator for the servos to separate the servo and raspberry pi power supply.



Preparation work: listing components, their sizes, features, etc.

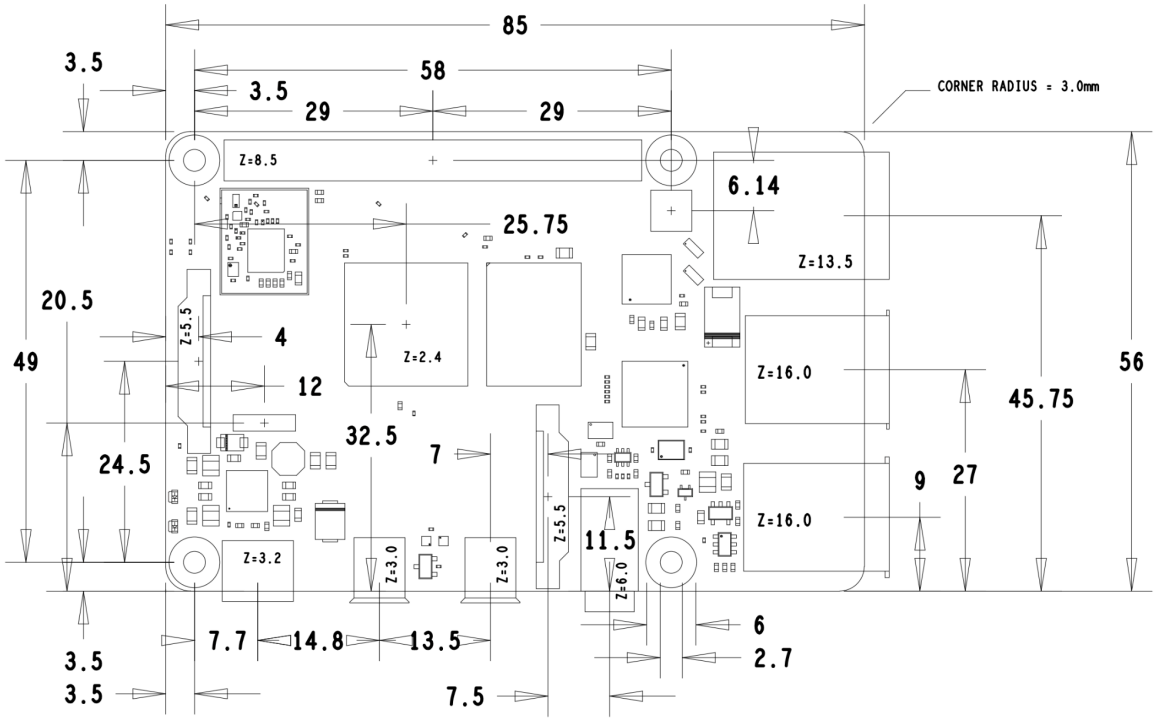
- 20x2 Connection: Surface mounted ⟶ through-hole
- LED vs Screen ⟶ LED (to show states)

- Capacitor — surface mounted, package 0805, current: C_Disc_D4.3mm_W1.9mm_P5.00mm
- Add connections for the camera gimbal – 1 signal pin + GND + V
- Shift components to the right of raspberrypi connector so that the components are over the raspberrypi + add mounting holes
    - Find dimensions of raspberrypi
    - Find LCSC part numbers
    - Remove all connectors, button, terminal screw, 2 of the capacitors (1, 0.1) from position file
    - Correct orientation of parts
        - Diode – plus on open side, minus on closed side
    - https://cart.jlcpcb.com/quote?rand=0.35969832612519403

LCSC	**Designator**	**Footprint**	**Quantity**	**Value**
	C1	CP_Radial_D10.0mm_P5.00mm	1	470uF
C59461	**C2**	CP_Radial_D5.0mm_P2.50mm	1	22uF
C49678	**C3**	805	1	0.1uF
C28323	**C4**	805	1	1uF
C2286	**D1**	603	1	Red_LED
C2297	**D2**	805	1	Green_LED
C2296	**D3**	805	1	Yellow_LED
Extended - C709041	**J1**	TerminalBlock_Phoenix_MKDS-1,5-2-5.08_1x02_P5.08mm_Horizontal	1	Screw_Terminal_01x02
	J2	PinHeader_1x06_P2.54mm_Vertical	1	Motor_Encoder_Conn_01x06_Pin
	J3	PinHeader_2x20_P2.54mm_Vertical	1	Raspi_Conn_02x20
	J4	PinHeader_1x04_P2.54mm_Vertical	1	ComPass_Conn_01x04_Pin
	J5	PinHeader_1x04_P2.54mm_Vertical	1	Lidar_Conn_01x04_Pin
	J6, J8	PinHeader_1x03_P2.54mm_Vertical	2	Servo_Conn_01x03_Pin
	J7	PinHeader_1x08_P2.54mm_Vertical	1	Gyro_Conn_01x08_Pin

C17617	**R1, R2**	805	2	300
C17471	**R3**	805	1	150
	SW1	Tact_Switch_TL59AF160Q_E-Switch	1	SW_Push
	U1	DIP-8_W7.62mm	1	~
	U3	Voltage_Regulator_MP1584	1	~

Source:
https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008343-DS-1-raspberry-pi-4-mechanical-drawing.pdf?disposition=inline



GPIO pins	
24	Servo for camera
13	Red
19	Green
26	Yellow

LED

We decided to set the LEDs to only a fraction of their full brightness as we only needed it to signal what mode the robot is in (obstacle, open, sleep) and we wanted to save power.

Green (0805)

½ Brightness — 5 mA → 2.6V

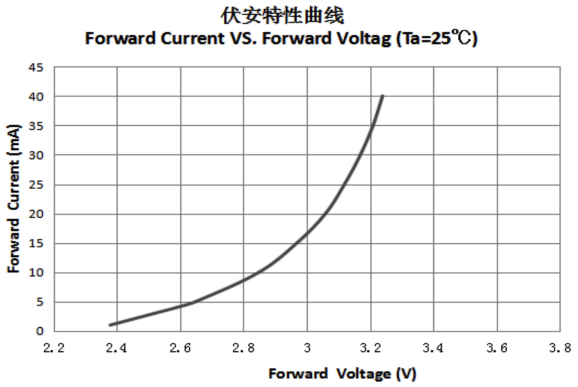
3.3V - 2.6V = 0.7V

Resistor 140 Ω

0805W8F1500T5E

Part No. C17471, Package 0805

-55℃~+155℃ 125mW 150V 150Ω Thick Film Resistor ±1% ±100ppm/℃ 0805 Chip Resistor - Surface Mount ROHS



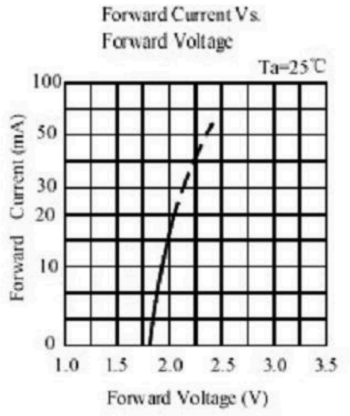
Red (0603) & Yellow (0805)

¼ Brightness — 5mA → 1.8V

5 mA = 0.005A < 16mA (GPIO pin maximum current)

3.3V - 1.8V = 1.5V

R = 300 Ω



Resistor:

0805W8F3000T5E Part No. C17617, Package 0805 Description -55℃~+155℃ 125mW 150V 300Ω Thick Film Resistor ±1% ±100ppm/℃ 0805 Chip Resistor - Surface Mount ROHS

Power & Sensors

Raspberry Pi vs ESP32

For this year's iteration of our robot, we chose to continue using the Raspberry Pi over the ESP32.

Raspberry Pi 4B	**ESP32**
More expensive (S$135 for the 4gb RAM model)	Cheaper (S$5-10 depending on model)
Runs OpenCV faster with native support for Raspberry Pi Camera	Slower processing due to limited RAM and a less powerful processor
Runs on 5V, which drains the battery quicker	Requires less power, extending battery life

Ultimately, we chose the Raspberry Pi as it can run image processing faster, preventing the possibility of missing out obstacles during the obstacle run. This year, we also used the Iterative Closest Point Method to perform localisation in the playfield, which is more computationally intensive. We had used it in last year's version of our robot, so we were more familiar with the platform.

Power

The robot is powered by two 3.0V 1650 mAh batteries. The batteries are directly connected to the PCB, which splits the power between the Raspberry Pi and the camera's servo motor.
A voltage regulator is placed before the DC Motor and the Raspberry Pi as it only takes 5V. Components such as the lidar, steering servo motor, gyro and LEDs are connected to the Raspberry Pi via the PCB and are powered by its 3.3V output pins. The LEDs are not running at their maximum current, as we do not need it to be at its maximum brightness.

The camera's servo motor uses a separate voltage regulator from the Raspberry Pi and DC Motor, as we found that it drew a significant current when the camera's servo motor moved, causing lag which affected the Raspberry Pi Camera's image processing.

Overall, the robot is able to run continuously for around 2 to 2.5 hours before the motors start to slow down due to the dip in voltage.

Sensors

Lidar

Our choice of lidar is the CoinD4, which is a small and relatively affordable Time of Flight (ToF) sensor. It has a close offset, making it more consistent at detecting corners. The faster update rate (10Hz) improved the accuracy of localisation by providing more frequent data points.



We carried out some tests to find out the disparity between the distances of black and white walls from the lidar and the actual distance, which turned out to be fairly small (average of 1.11cm), so no correction was needed.

White			
Actual Distance	[White] Proposed Distance	Error	[Black] Proposed Distance
3	3.6	0.6	
5	5.2	0.2	
10	10.6	0.6	
20	20.7	0.7	
30	30.9	0.9	30.6
49.6	51	1.4	50.5
62.7	64.3	1.6	63.8
75.5	76.8	1.3	
88.4	90	1.6	89.5
111.3	112.8	1.5	
120	121.4	1.4	
150	151.5	1.5	
170	171.3	1.3	
200	200.9	0.9	201
230	231.2	1.2	230.5
250	251.2	1.2	
270	271.1	1.1	
300	301	1	
		1.111111111	

Previously we had considered using the LDS02RR lidar, which is considerably larger than the CoinD4. During our testing, we encountered several major issues:

1. The sensor and laser are far apart, and neither are in the centre, so it tended to miss the corners.
2. It has a slower update rate (5 Hz), meaning that we would have fewer lidar readings to work with, providing less frequent data points for localisation.
3. It had many failed or "0" readings per cycle, especially when objects are close to the lidar due to the large separation between the sensor and the laser.



Operating voltage / Motor: 3.6V (counterclockwise rotation)
Data Output:

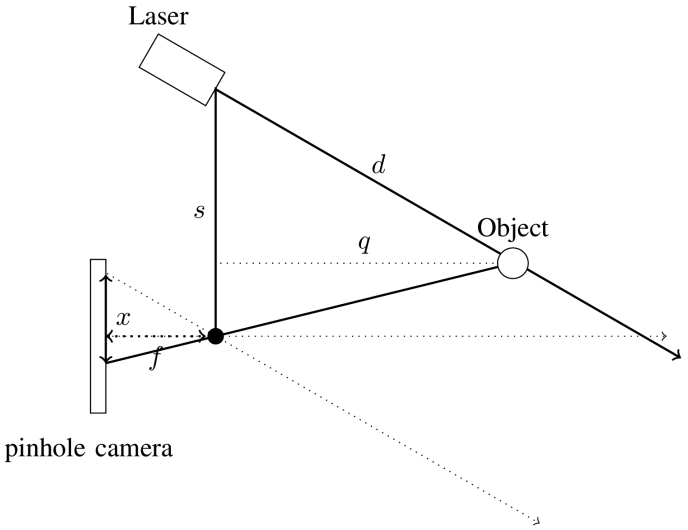
- 90 frames per cycle, with 4 readings per frame.
- This gives about 189 readings for 90 degrees

Wiring pinout:

- Red - Motor (+)
- Black - Motor (-)
- Brown - GND
- Yellow - 5V
- White - Tx

90 degrees: reading 189
90 frames – 4 readings each

Triangulation (not using pinhole camera)



- As the object is further away from the the laser, it moves towards the right and vice versa
- Camera and laser are not directly parallel (as the laser point will never go beyond the center to the right side of the camera) → slightly angled in order to use the full range of the camera

Tested the lidar using a similar procedure to this paper:
https://www.diva-portal.org/smash/get/diva2:995686/FULLTEXT01.pdf

Gyro/Compass

Our robot uses the MPU6050 gyro, which can measure rotation around 3 axes, but we only use the yaw axis for our application. It is reliable and accurate when measuring rotation over short periods of time, which is suitable for our robot as it only runs for a maximum of 2 to 2.5 minutes at a time, keeping our odometry accurate.

Before implementing the gyro to our robot, we tested and calibrated it by putting it on a rotating board to find its scaling error.

We also considered using a compass, however it is prone to interference from objects with a magnetic field, which causes the magnetic field to curve. This results in the errors becoming non-linear and harder to correct. While it does not drift unlike gyros, it is much harder to calibrate. Hence, we chose to use a gyro for our robot.

Camera

During our preparation, we experimented with different modules and configurations of the Raspberry Pi Camera.

The first configuration was the Raspberry Pi Camera Rev 1.3 without any modifications to it. We measured its Field of View (FOV) to be around 72 degrees. We did so by placing two objects at the two ends of the camera's view and drawing a line to the centre of the camera, then we measured the angle between the two lines. This configuration was flawed as the camera could not see the upcoming obstacles at corners due to its limited FOV. In order to circumvent this issue, the robot had to perform awkward turns to face the obstacles, which wasted a lot of time during the runs.

In the second configuration, we used the Raspberry Pi Camera Wide Angle Module, which had an FOV of 120 degrees. Although it has a much wider FOV, the robot still had to turn to face the obstacles. The wide angle lens also resulted in a fish eye effect, and made the obstacles look very small, forcing us to lower the minimum blob detection size for detecting obstacles. This increased the frequency of false positives, so we did not go with this configuration.

The third configuration used a servo motor to swivel the Raspberry Pi Camera Rev 1.3, allowing it to turn to face upcoming obstacles at the corners, which it could not detect easily previously. It also allowed us to programme the servo motor to track the position of the obstacles as their positions are fixed.

For the first two configurations, we 3D printed physical camera croppers to prevent any red or green coloured objects outside the playfield from being detected. In our third and final configuration, we cropped the image of the camera in code as it can be easily changed.

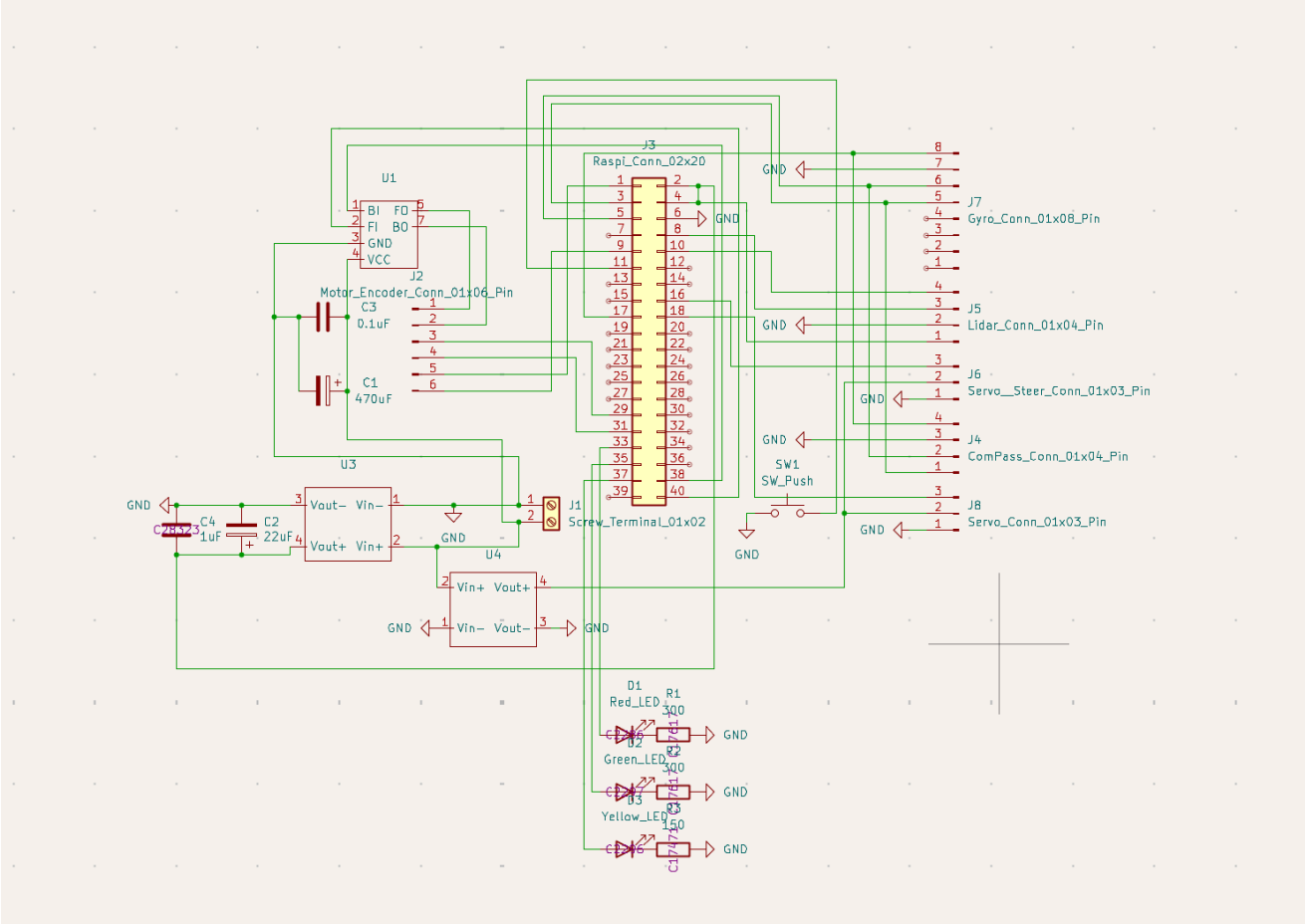
When processing images from the camera, we calibrated the camera in a few ways:

1. The image is converted from BGR to HSV colour space, as it is easier and more reliable to isolate colours under varying lighting conditions.
2. Gaussian blur is applied to the image to smoothen out any uneven surfaces and remove noise to aid obstacle detection.
3. The resolution was also set to VGA as a lower resolution can increase the speed of image processing.
4. Minimum area and pixel sizes were set so that tiny blobs of red or green are discarded, preventing false positives.

Whenever there is a change of location, we also have to re-calibrate the thresholds as the shade of red and green may change. Hence, we coded a calibration programme

Wiring diagrams

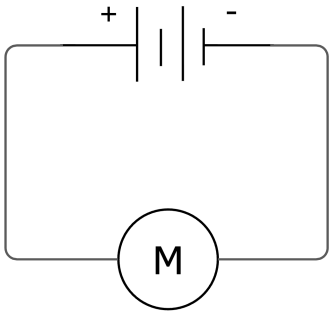
PCB



External connections

Raspberry Pi ↔ Raspberry Pi Camera

Power supply for motor: Multi-cell Battery ↔ DC motor



Design History

2025

2 March 2025

- Selected an RC car as the reference for the robot chassis
- Selected the Raspberry Pi as the main computing platform
- Selected LEGO Spike Prime wheels (56 × 14 mm)
- Selected the existing magnetic encoder integrated into the drive motor
- Explored LEGO, 3D-printed parts, and wood for the chassis and differential

9 March 2025

- Selected Ackermann steering for the front wheels
- Designed the Ackermann steering linkage
- Laser-cut a wooden prototype before using acrylic

16 March 2025

- Modified the chassis to accommodate the front wheels and servo motor
- Added a slot for mounting the steering servo
- Modified the Ackermann beam to fit the servo

23 March 2025

- Connected the drive motor to an H-bridge
- Established serial communication with the Raspberry Pi
- Used I^2C communication for the gyroscope and compass

30 March 2025

- Implemented motor control using PWM frequency and duty cycle
- Added speed control through the drive function
- Replaced the existing H-bridge
- Implemented motor stopping control

6 April 2025

- Crimped and connected encoder wires
- Developed the servo steering function
- Defined the steering command range around 1500 µs, with negative values for left steering and positive values for right steering
- Added a slot to the Ackermann beam

- Added a voltage regulator to power the Raspberry Pi from the battery

13 April 2025

- Integrated and calibrated the MPU6050 gyroscope
- Created a gyroscope library using SMBus
- Compared gyroscope and compass heading measurements

20 April 2025

- Integrated and wired the compass
- Implemented compass calibration using linear interpolation
- Finalised and cut the base plate from acrylic

27 April 2025

- Assembled the robot base and mounted the sensors and Raspberry Pi
- Added gyroscope correction to the steering function
- Designed a second plate for the LDS02RR LiDAR

4 May 2025

- Calibrated encoder distance:
    - 0.298 mm per step in the first test
    - 0.296 mm per step in the second test
- Modified the front-wheel mounts using longer connectors and stronger LEGO components
- Adjusted the steering mechanism to prevent wheel interference

11 May 2025

- Repaired the encoder wiring

18 May 2025

- Tested the LDS02RR LiDAR measurement system
- Investigated LiDAR triangulation and distance measurement
- Designed a new mounting plate for the CoinD4 LiDAR
- Tested LiDAR distance accuracy

25 May 2025

- Tested the CoinD4 LiDAR
- Replaced the original LiDAR mounting arrangement with a temporary holder and revised level plate
- Tested the drive motor and encoder step direction

1 June 2025

- Tested a generic USB webcam
- Implemented blob detection
- Calibrated red and green colour detection
- Developed socket-based transmission of LiDAR points to a computer
- Used Matplotlib to visualise LiDAR measurements

8 June 2025

- Implemented LiDAR spike detection
- Implemented Cartesian coordinate conversion for detected spikes
- Implemented landmark matching using a distance threshold
- Added gyroscope calibration saving to a text file
- Adjusted LiDAR processing to handle zero readings and full-revolution angle wrapping

13 June 2025

- Implemented LiDAR visualisation using polar coordinates
- Implemented Cartesian visualisation of detected spikes and matched landmarks
- Implemented the initial estimate_pose() function
- Developed initial-position detection for the Open Challenge

15–16 June 2025

- Completed the spike landmark program
- Implemented proportional steering control using steer_in_dir()
- Implemented path augmentation using augment_path() and augment_paths()
- Implemented path-following functions using drive_path() and drive_paths()
- Developed pose estimation using straight-line and circular-arc movement models
- Implemented landmark matching and Cartesian coordinate conversion
- Implemented position and heading correction using filter ratios
- Implemented position and heading merging to compensate for odometry and gyroscope drift

22 June 2025

- Integrated SLAM-related code from GearsBot
- Investigated missing LiDAR readings around corners
- Adapted spike detection to account for zero-distance measurements

29 June 2025

- Tested the navigation code
- Revised the PCB schematic
- Connected the 5 V and ground pins
- Adjusted the PCB component layout

6 July 2025

- Attempted a third PCB milling iteration
- Revised the PCB dimensions to fit the Raspberry Pi

13 July 2025

- Tuned the position filter ratio used to combine odometry and landmark measurements
- Reduced the influence of landmark corrections when the discrepancy exceeded 100 mm
- Investigated unexpected LiDAR spikes

20 July 2025

- Completed soldering the electronic components onto the PCB
- Tested PCB fit, electrical continuity, and voltage
- Added mounting holes for the Raspberry Pi and battery holder

27 July 2025

- Redesigned and recut the top acrylic plate
- Mounted the PCB
- Corrected the direction of the displacement calculation
- Updated pose estimation to account for the offset between the LiDAR and rear-wheel positions
- Adjusted gyroscope timing and pose estimation

3 August 2025

- Integrated the PCB with the Raspberry Pi, gyroscope, servo, LiDAR, motor, and encoder
- Investigated Raspberry Pi power and connectivity issues
- Added 3D-printed spacers to level the PCB

10 August 2025

- Added a LiDAR distance filter to ignore readings below 250 mm
- Investigated duplicate landmark matches
- Investigated gyroscope scaling error
- Improved PCB soldering at the 5 V connections

17 August 2025

- Repaired a PCB solder connection
- Added 16 landmarks for the Open Challenge
- Created a new outer navigation path
- Adjusted the robot's rotation rate
- Implemented start-button input using GPIO and pull-up configuration

- Configured the Raspberry Pi to connect through a portable router
- Added 24 obstacle-related landmarks
- Implemented separate inner and outer paths
- Added navigation decision points
- Integrated camera-based colour detection with navigation
- Investigated camera field of view and path-selection behaviour

24 August 2025

- Added a path-count check to stop the robot after completing its route
- Corrected the gyroscope Z-axis calibration calculation
- Investigated gyroscope scaling calibration using known rotation angles

12 October – 23 November 2025

- Developed the parallel parking procedure
- Refined the parking alignment sequence
- Adjusted the robot's reversing movements
- Modified the first alignment stage to occur farther from the parking wall
- Adjusted the second alignment stage to bring the robot closer to the wall
- Corrected path augmentation
- Renamed navigation paths for consistency

26 November 2025

- Added LiDAR buffer clearing after the start-button press
- Corrected path augmentation
- Revised the navigation path names
- Reduced the distance of the first two reversing movements
- Updated the two-stage parking alignment procedure

2026

10 May 2026

- Redesigned the robot frame to reduce its size
- Redesigned the PCB and reorganised the electronics
- Redesigned the differential
- Redesigned the wheel components
- Repositioned the camera onto a motor-mounted arrangement
- Centred the steering servo
- Shifted the Raspberry Pi and battery holder towards the left side
- Investigated alternative motor mounting orientations

- Redesigned the steering linkage
- Developed a custom steering-related component to maintain a flat chassis base

17 May 2026

- Redesigned the PCB
- Sent the PCB for manufacturing
- Designed and 3D-printed camera mounts
- Redesigned the differential
- Shifted the Raspberry Pi and battery holder to improve weight distribution

24 May 2026

- Redesigned the wheel components
- Redesigned the Ackermann steering linkage
- Redesigned the top plate

14 June 2026

- Added liftarm bracing beneath the steering linkage
- Added standoffs to limit Ackermann steering movement
- Connected PCB ground pins
- Assembled the top plate
- Continued point-cloud transformation and localisation testing

21 June 2026

- Resoldered the PCB
- Assembled the top plate
- Implemented and tested point-cloud transformation
- Tested the refactored localisation code

28 June 2026

- Replaced the steering servo
- Cut and assembled the acrylic base
- Tested the refactored code
- Tested the ICP localisation algorithm
- Set a target ICP execution time of less than 0.1 seconds

5 July 2026

- Tested the refactored code
- Tested hardware components
- Redesigned the PCB
- Tuned the camera and encoder

- Added a LiDAR buffer-clearing function
- Sent the PCB for manufacturing

12 July 2026

- Tested the refactored code
- Tested ICP execution time against the target of less than 01 seconds
- Tuned the encoder
- Cropped the top portion of the camera image

19 July 2026

- Assembled and soldered the new PCB

26 July 2026

- Assembled the new PCB
- Repaired the LiDAR wiring
- Tested initial-pose detection in clockwise and counterclockwise configurations
- Fixed the loose shaft
- Tested the Open Challenge run programme
- Tested ICP using deliberately incorrect initial poses
- Added LiDAR buffer clearing
- Worked on PID speed control
- Investigated reverse-voltage protection and safe Raspberry Pi shutdown

2 August 2026

- Fixed the initialisation script
- Added start-button control and LED indicators
- Tested the Obstacle Challenge run programme
- Tested ICP with deliberately incorrect initial poses
- Added a LiDAR buffer-clearing function
- Checked the start-button connection on GPIO17
- Worked on PID speed control
- Investigated reverse-voltage protection and safe Raspberry Pi shutdown
- Began redesigning the PCB power distribution
- Investigated a separate voltage regulator for the servos

8 August 2026

- Investigated gyroscope angle error and LiDAR coordinate transformations
- Developed camera tracking
- Tested camera colour detection
- Modified obstacle paths and obstacle configurations

- Redesigned the PCB with a larger voltage regulator
- Added a separate regulated power supply for the servos

16 August 2026

- Investigated the cause of angle error
- Modified obstacle paths and obstacle configurations
- Continued parallel parking development

23 August 2026

- Crimped the servo wire
- Soldered the PCB
- Tested the counterclockwise Obstacle Challenge route

30 August 2026

- Tuned the estimated pose
- Added ICP testing code for stationary-robot tests and manually altered initial poses
- Added wall-related debugging and print statements
- Tested different obstacle configurations
- Tested starting from the parking position

5 September 2026

- Implemented double buffering for LiDAR measurements
- Modified the CoinD4 frame parser to continue reading after a frame was completed
- Added measurement buffering to reduce LiDAR data lag
- Investigated threading for camera processing
- Added the extended inner wall to the Open Challenge map
- Modified the counterclockwise parking procedure
- Added a short path to the obstacle route
- Modified the parking path to move farther from the walls
- Investigated whether parking localisation should use ICP wall matching or rely on odometry
- Modified the parking movement sequence to include backward, short forward, and short backward movements
- Added clockwise and counterclockwise obstacle route configurations

6 September 2026

- Measured the main-loop performance with camera processing enabled and disabled
- Recorded approximately 28.2 loops per second with camera processing enabled
- Recorded approximately 1,253.8 loops per second without camera processing
- Evaluated the effect of camera processing on LiDAR measurement buffering
- Tested buffered LiDAR acquisition during camera processing

17 October 2026

- Added an additional path for the Open Challenge shortcut
- Modified the shortcut to skip a path segment
- Analysed the Ackermann steering mechanism
- Reviewed spare hardware and equipment requirements

24 October 2026

- Planned clockwise Open Challenge shortcut testing
- Planned comparison of actual Ackermann steering behaviour with the intended steering geometry during parking